AI VIET NAM
@aivietnam.edu.vn

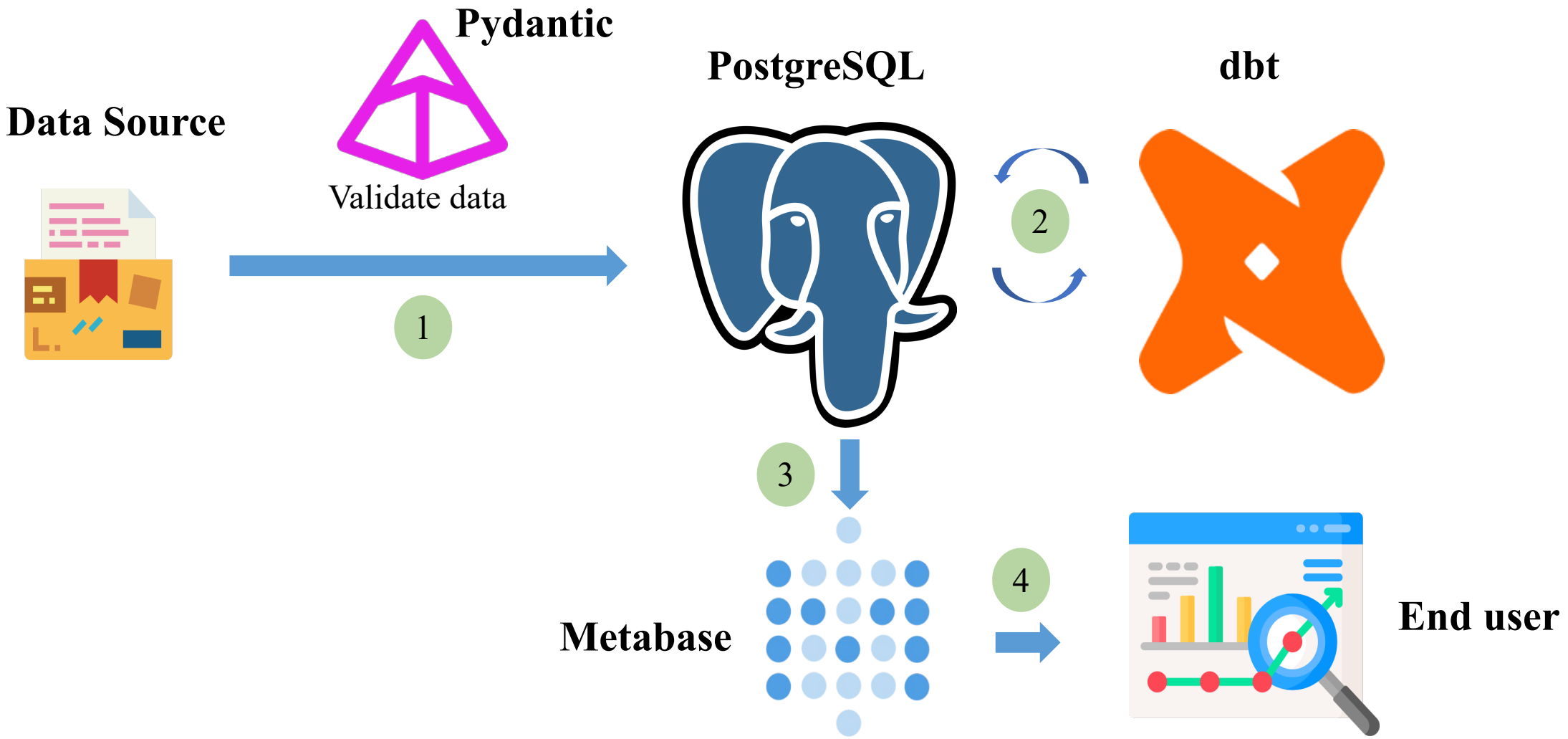
Data Pipeline and Orchestration using Dagster

Thuan Duong – Senior TA
Anh Duong – STA

Year 2026 Code & Data Proofreaders: Lê Quốc Thái- QA/QC: Nguyễn Thông

Objectives

❖ Pipeline



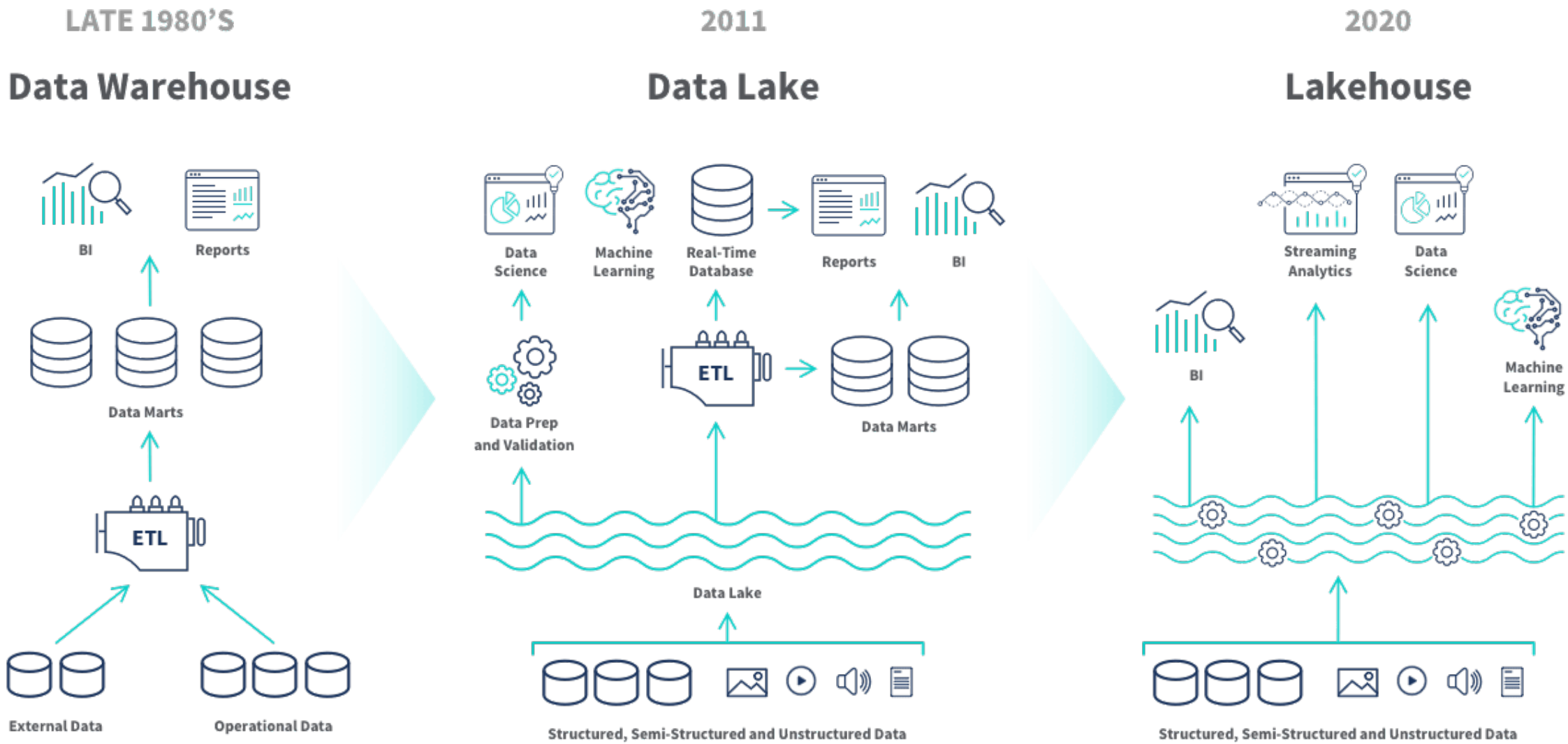
Outline

- Problems
- Data Pipeline
- Build with Dagster
- Deployment
- Questions

Problems

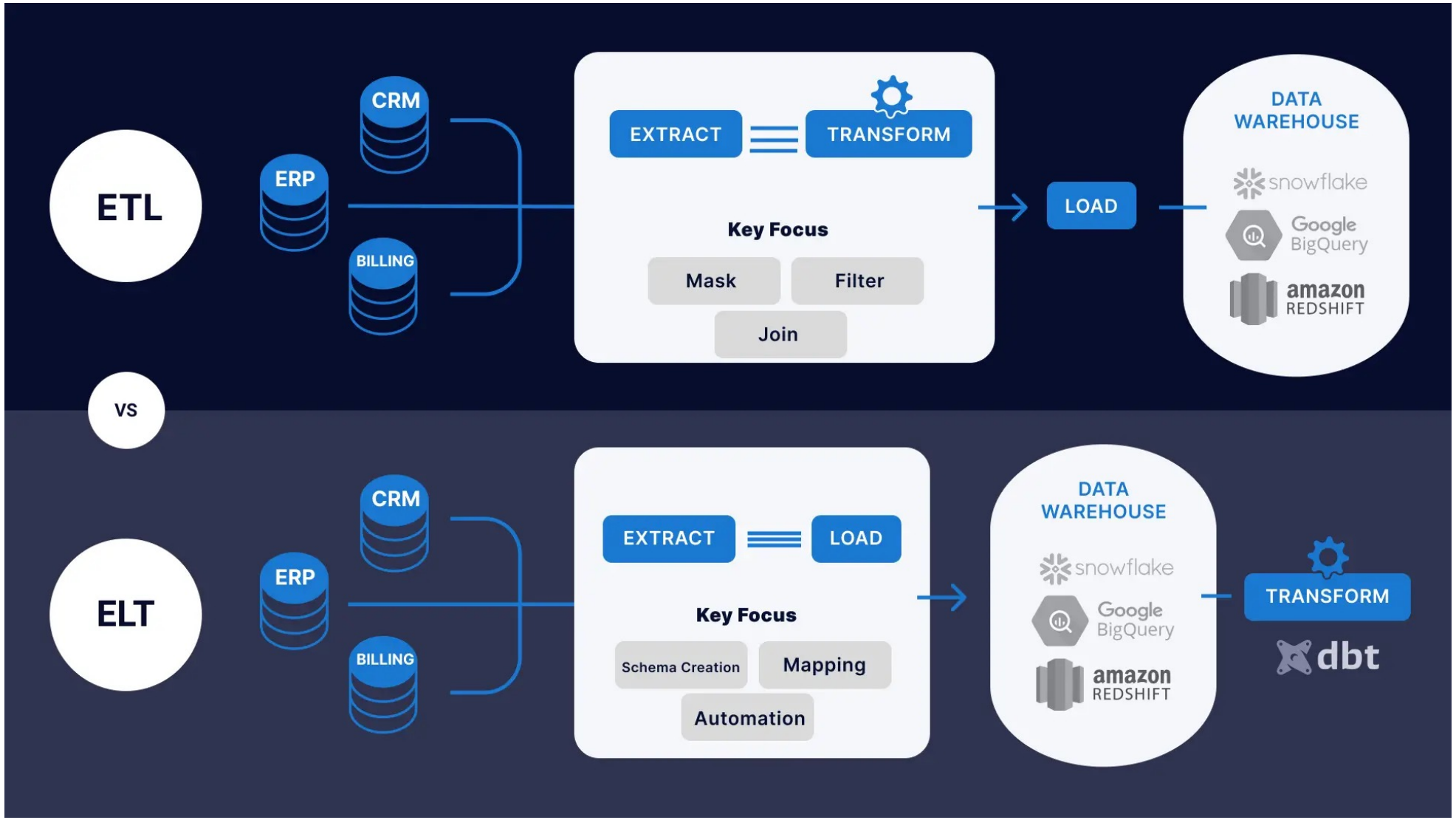
Problems

❖ Getting Started



Problems

❖ From ETL to ELT



Challenges

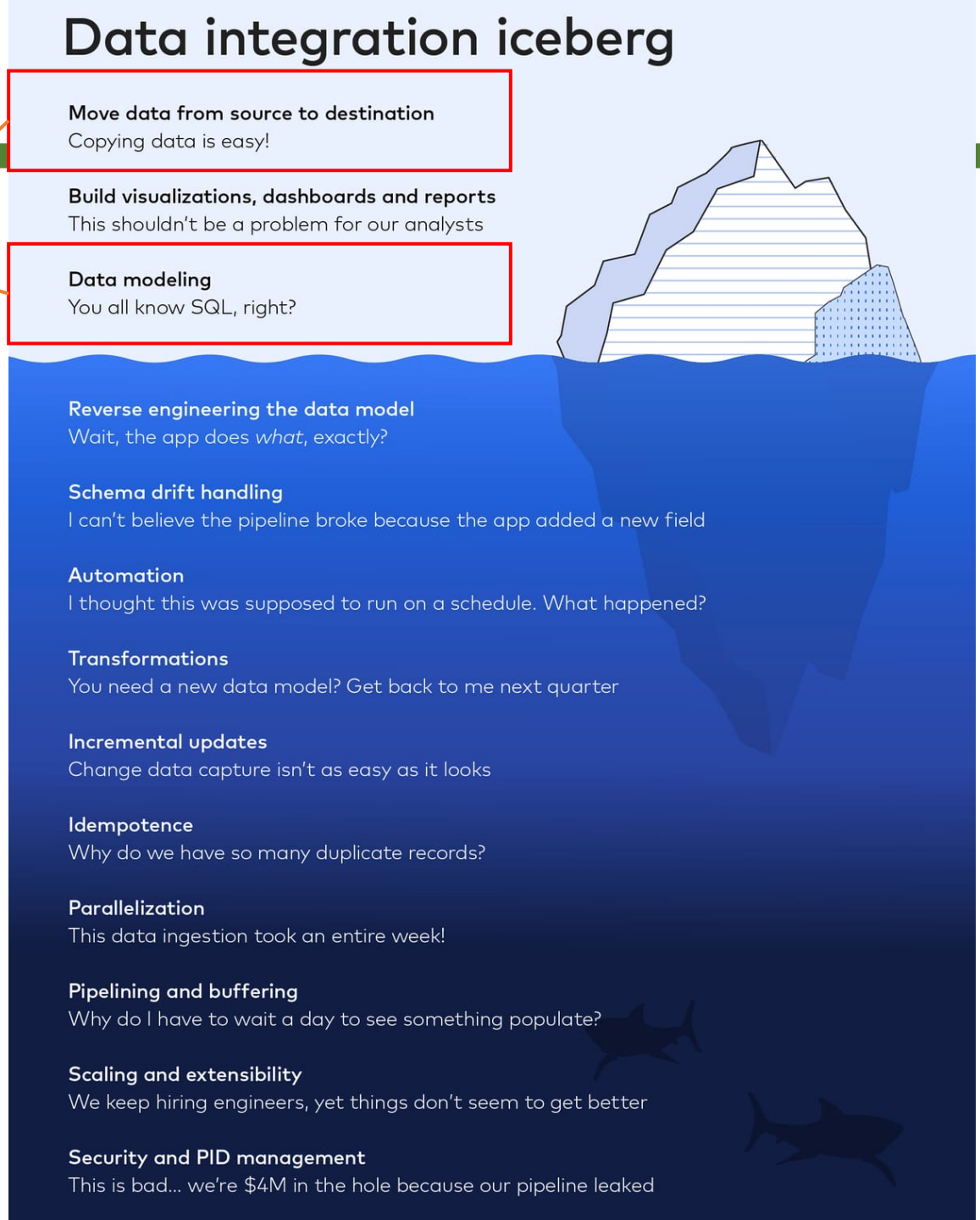
❖ Iceberg

We DONE!!

Volume, Variety

Build | VS | Buy

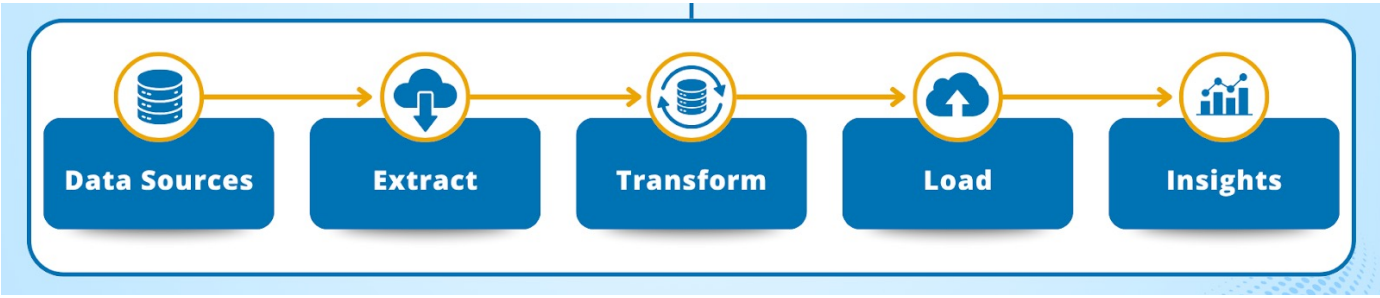
Data



Challenges

❖ Reverse engineering, drift

Reverse-engineer the source application's logic due to insufficient documentation or unclear structure.



Data pipelines break when source applications automatically add, remove, or modify data fields.



Challenges

❖ Automation & Transformations

Automated pipeline scheduling can lead to silent failures, stalled processes, or unexpected crashes.



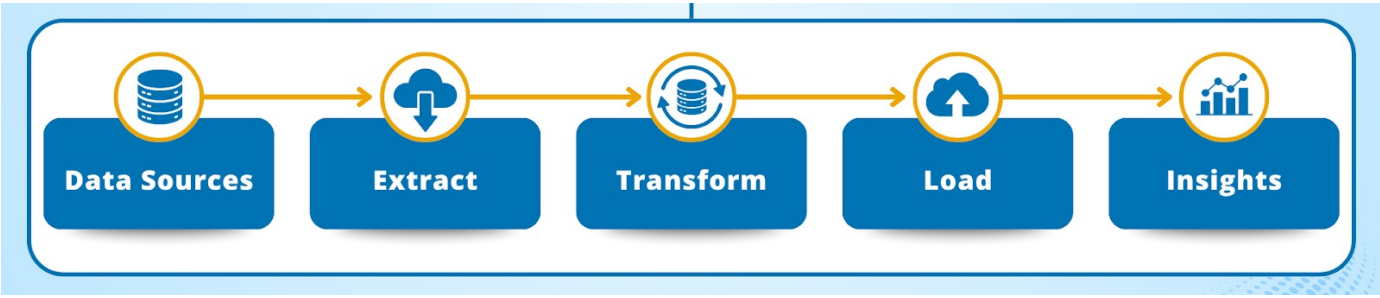
Processing and transforming raw data into an analysis-ready format requires complex logic and can be time-consuming.



Challenges

❖ Incremental & Idempotence

Synchronizing only changed data (Change Data Capture — CDC) is much more challenging than reloading the entire dataset.



Ensure **idempotency**: rerunning the pipeline produces consistent results without creating duplicate records.



Challenges

❖ Parallelization, Pipeline & Buffering

Optimize parallel processing for large datasets; otherwise, data ingestion can take weeks.



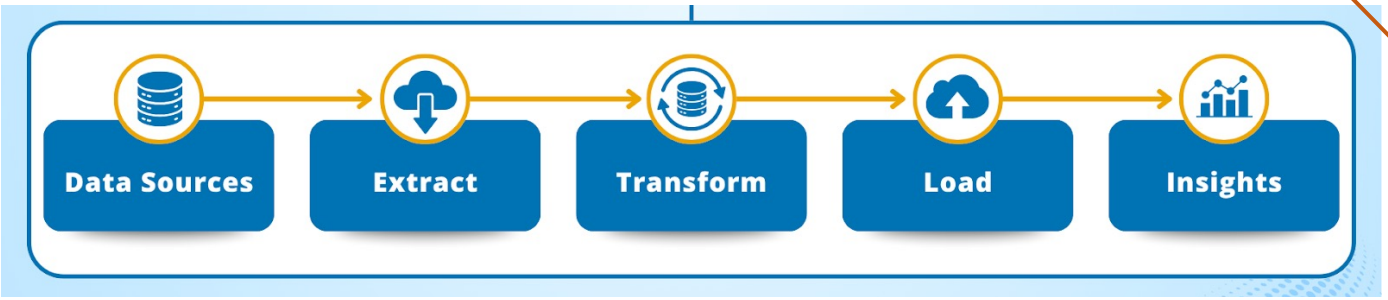
Manage data flows and buffering to ensure data is updated on schedule or near real time.



Challenges

❖ Pipelining & Scaling

Manage data flows and buffering to ensure data is updated on schedule or near real time.



As the system scales, complexity grows exponentially; adding more people does not immediately solve scalability challenges.

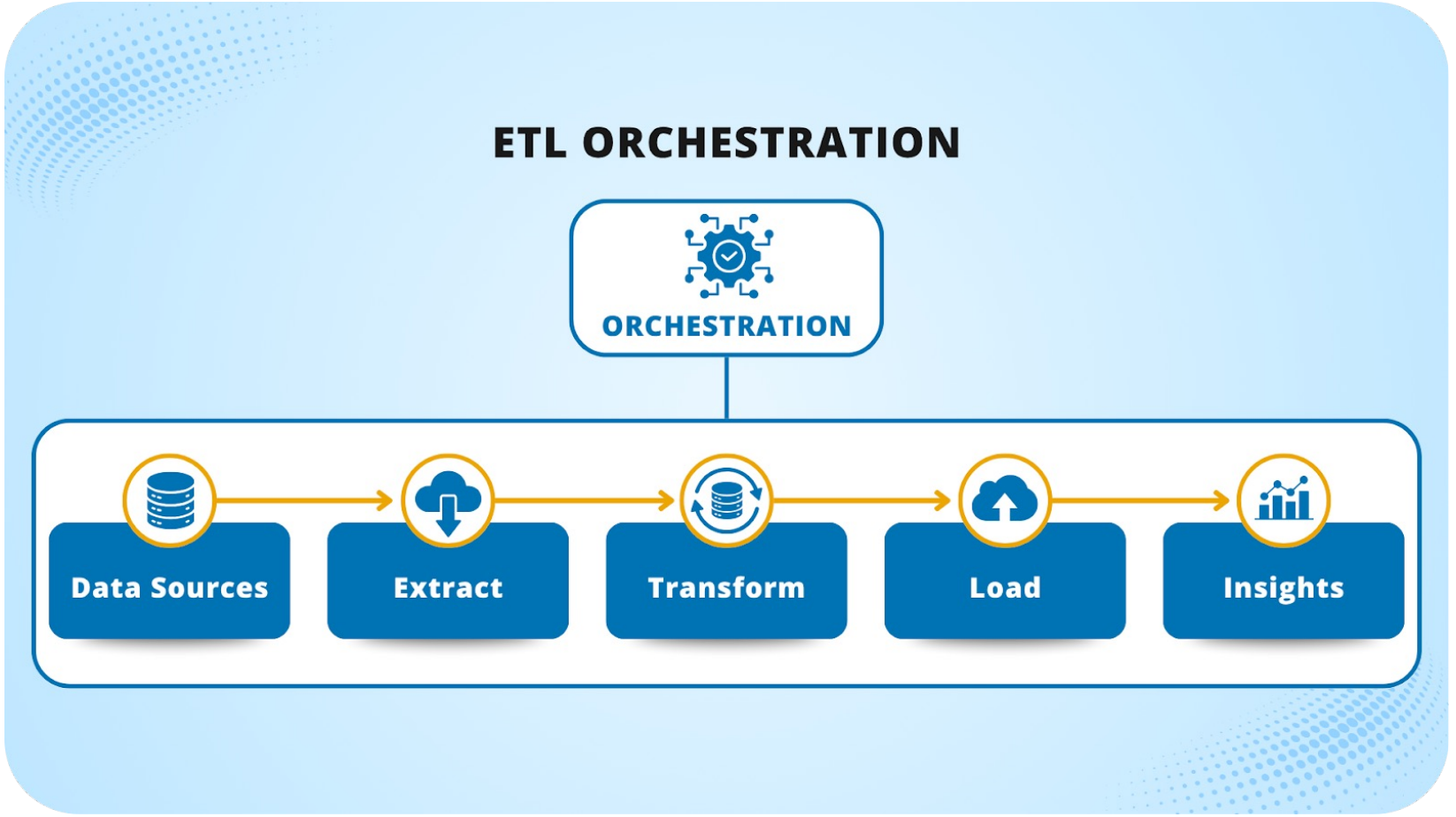


Data Pipeline

Data Pipeline

❖ Data Orchestration

Data Orchestration is the process of automating, scheduling, monitoring, and managing the execution order of data workflows.



Persistence

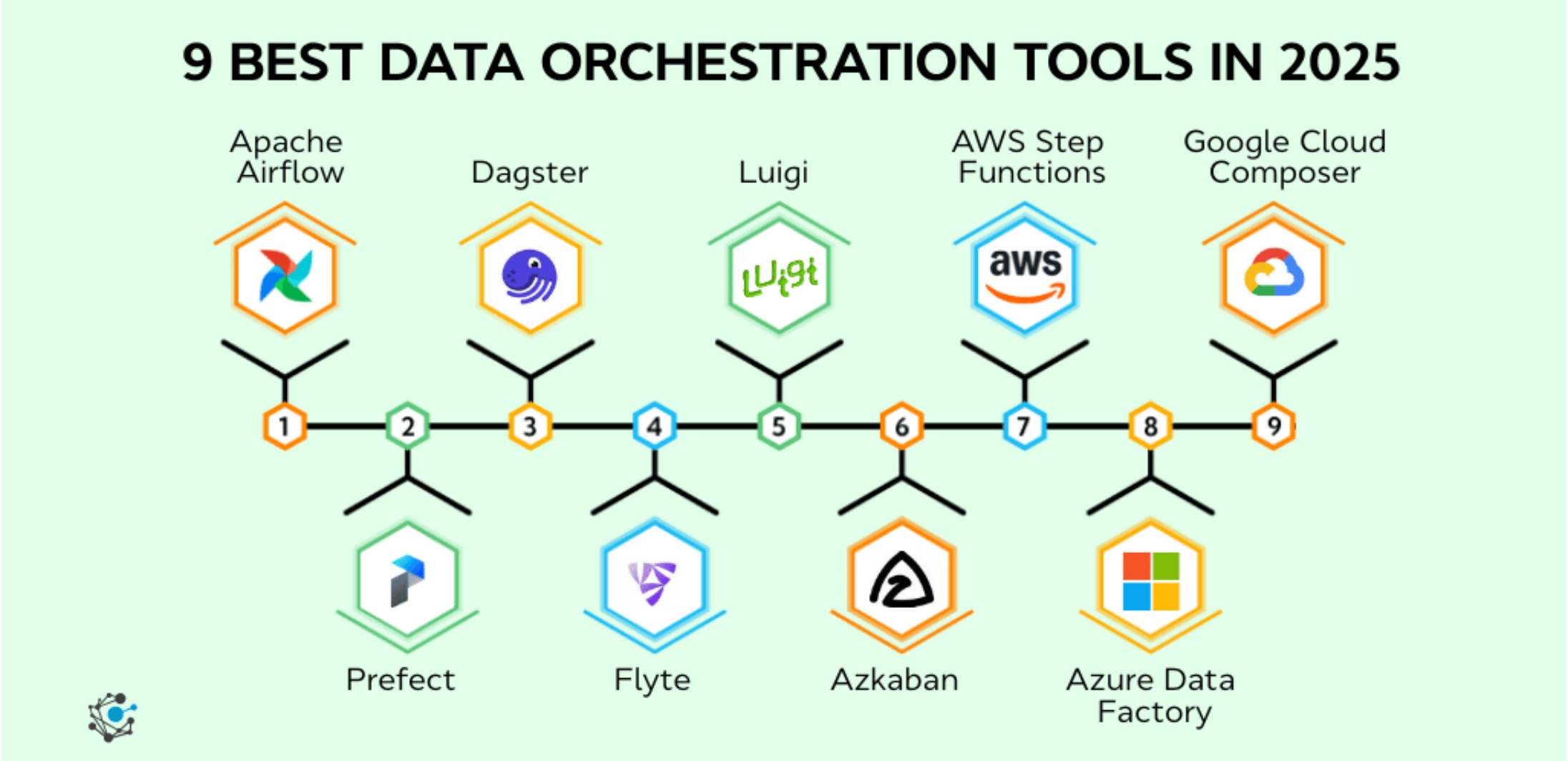
Error Handling

Status Reporting and Alerting

Metadata Storage

Data Pipeline

❖ Getting Started

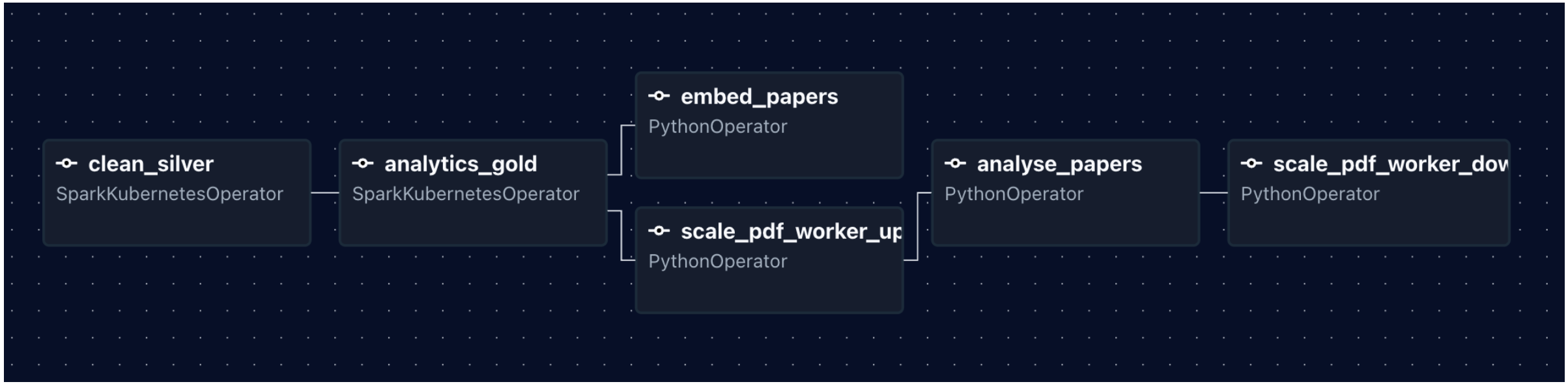


https://www.ovaledge.com/blog/data-orchestration-tools

Data Pipeline

❖ Task-Centric Orchestration (Airflow)

A pipeline can be viewed as a collection of scripts that need to be executed in a specific chronological order (e.g., using `BashOperator` and `PythonOperator`).



Limit

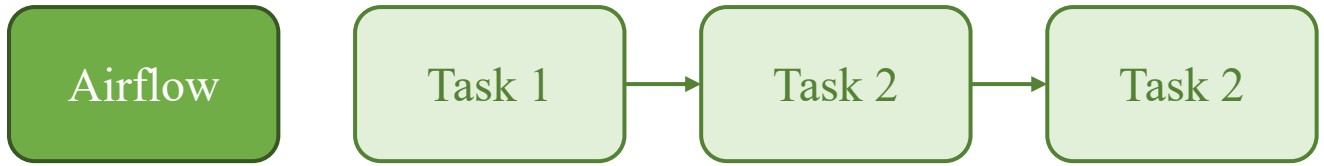
Tracks task success/failure only

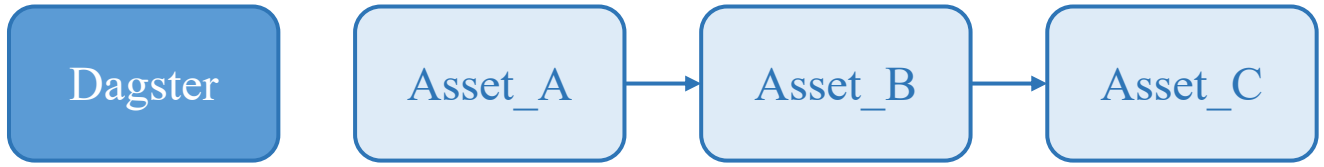
Does not track data state or quality.

Limited table/column-level lineage

Data Pipeline

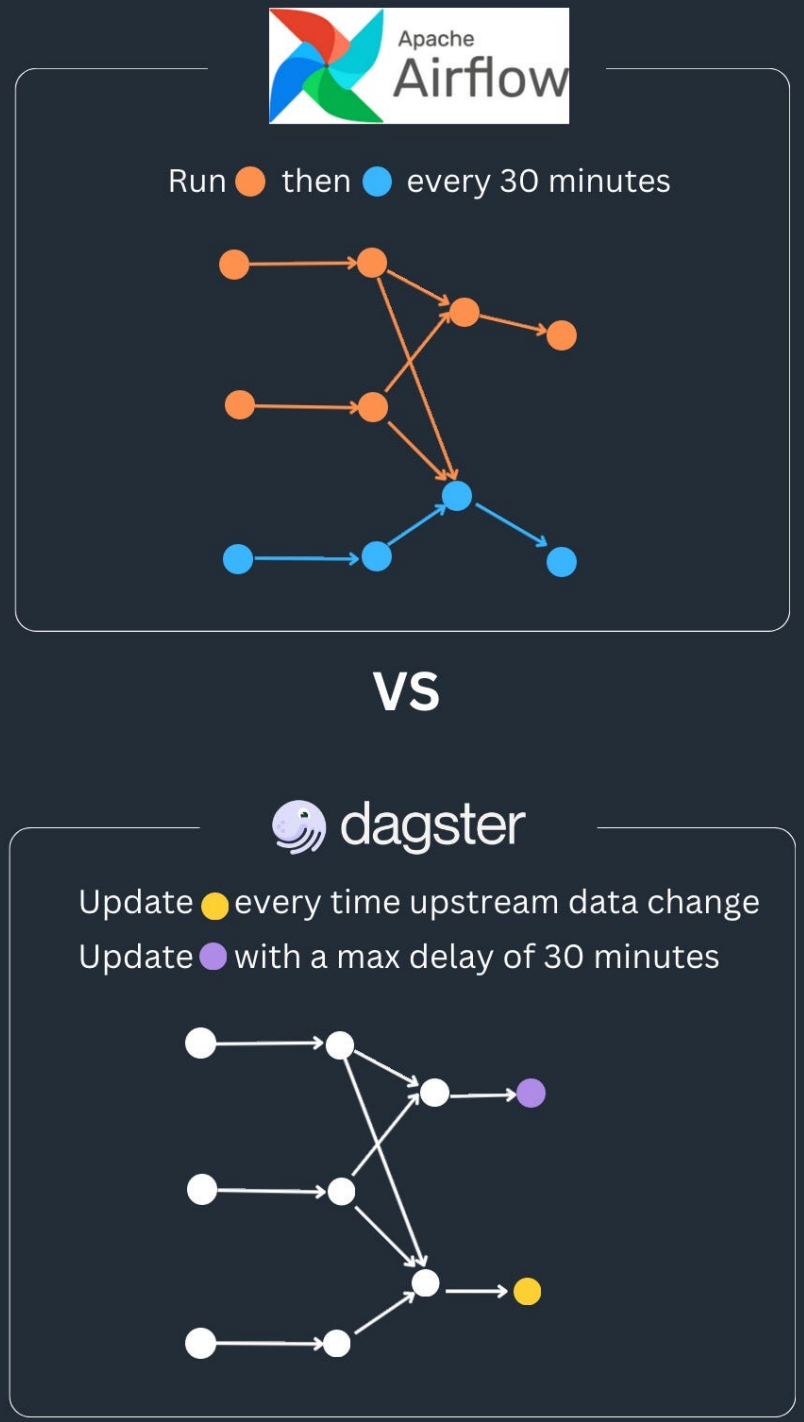
❖ Asset-Centric Orchestration





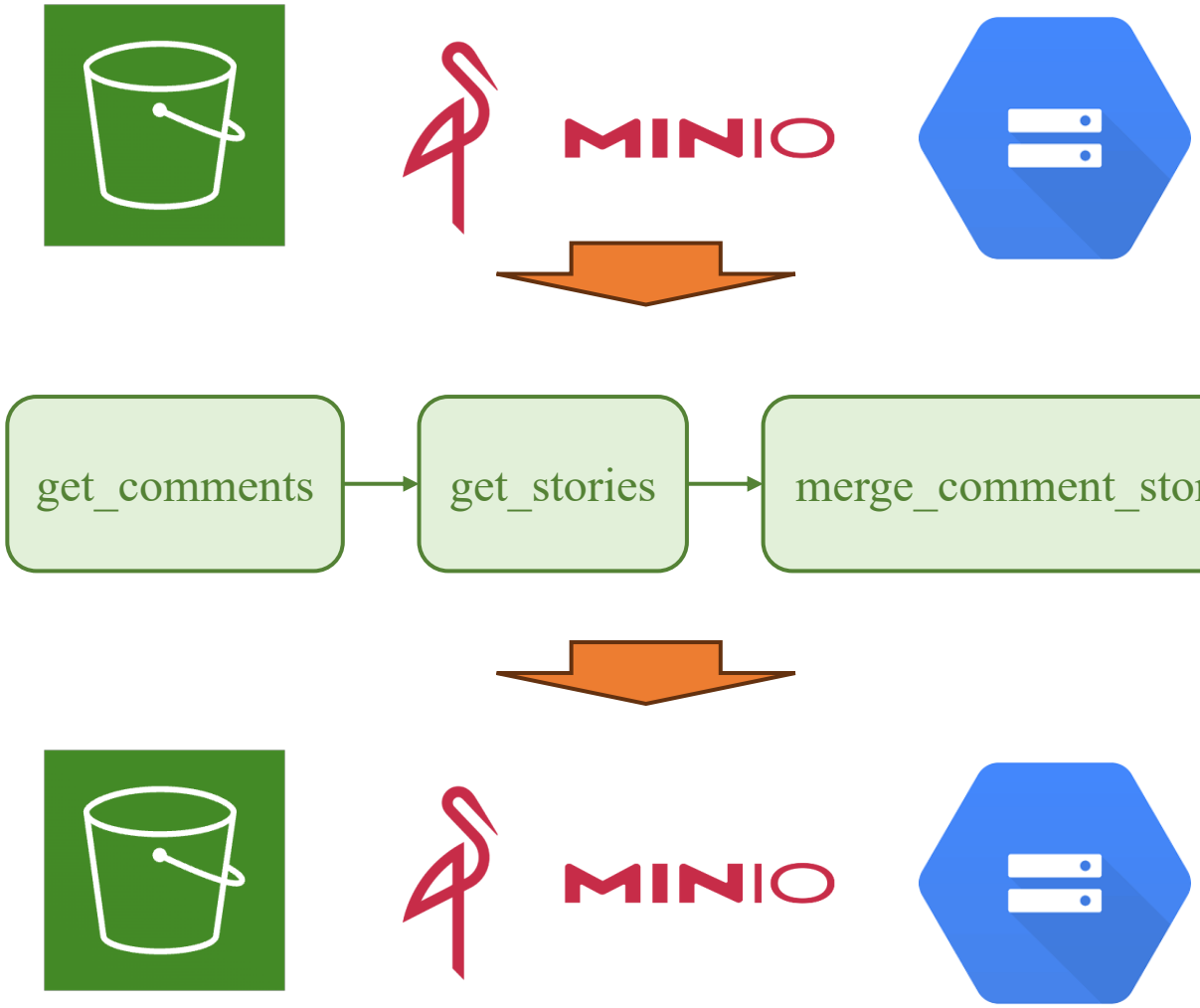
View pipelines as a network of interconnected Data Assets.
Answer three core questions:

- What is this data asset? (SQL table, S3 file, report)
- Which upstream data assets does it depend on?
- Is it up to date (Freshness)?



Data Pipeline

❖ Airflow code example




```python
from airflow.decorators import dag, task

@dag(
    dag_id="comment_stories_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule_interval="@daily",
    catchup=False,
    tags=["example", "airflow"]
)
def comment_stories_dag():
    @task # Task 1: Lấy danh sách comments
    def get_comments() -> list[dict]:
        return [
            {"comment_id": 1, "story_id": 100, "text": "Bài viết rất hay!"},
            {"comment_id": 2, "story_id": 100, "text": "Dagster trực quan quá."},
            {"comment_id": 3, "story_id": 101, "text": "Cảm ơn tác giả!"},
        ]

    @task # Task 2: Lấy danh sách stories
    def get_stories() -> list[dict]:
        return [
            {"id": 100, "title": "Giới thiệu về Data Pipeline"},
            {"id": 101, "title": "Xây dựng Data Pipeline hiện đại"},
        ]

    @task # Task 3: Kết hợp comments và stories
    def merge_comment_stories(comments: list[dict], stories: list[dict]) -> list[dict]:
        df_comments = pd.DataFrame(comments)
        df_stories = pd.DataFrame(stories)

        merged_df = pd.merge(
            df_comments,
            df_stories,
            left_on="story_id",
            right_on="id",
            how="inner"
        )
        return merged_df.to_dict(orient="records")

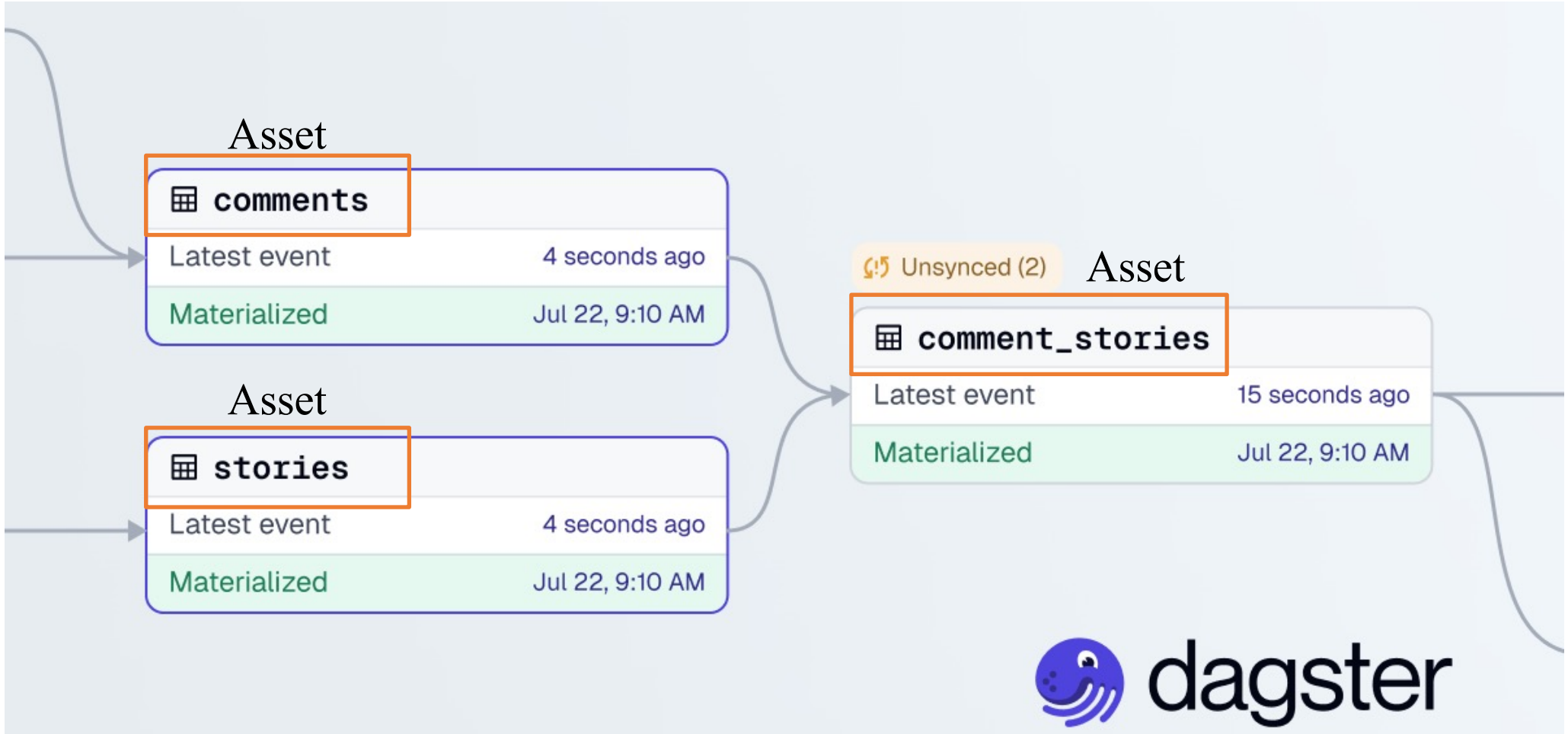
    comments_data = get_comments()
    stories_data = get_stories()
    merge_comment_stories(comments_data, stories_data)

comment_stories_dag()
```

Data Pipeline

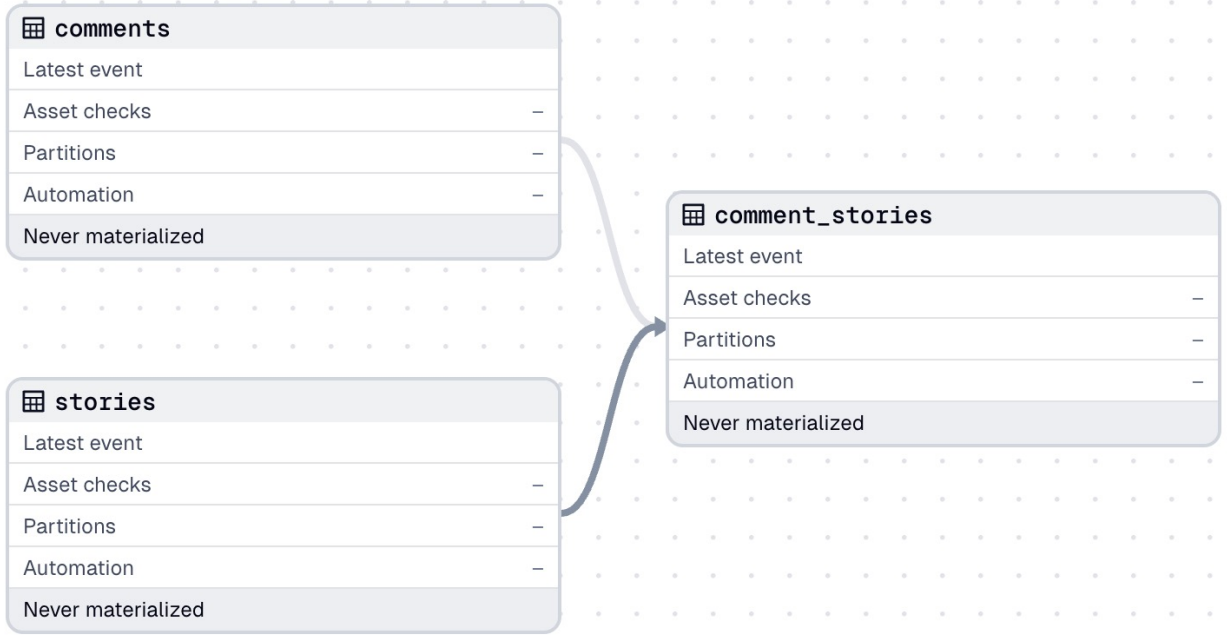
❖ Software-Defined Assets (SDA)

Software-Defined Assets are a new abstraction that **allows data teams to focus on the end products, not just the individual tasks**, in their data pipeline.



Data Pipeline

❖ Dagster code example




```python
import pandas as pd
from dagster import asset, Definitions

# 1. Asset comments (Lấy dữ liệu bình luận)
@asset
def comments() -> pd.DataFrame:
    """Asset chứa danh sách các bình luận."""
    return pd.DataFrame([
        {"comment_id": 1, "story_id": 100, "text": "Bài viết rất hay!"},
        {"comment_id": 2, "story_id": 100, "text": "Dagster trực quan quá."},
        {"comment_id": 3, "story_id": 101, "text": "Cảm ơn tác giả!"},
    ])

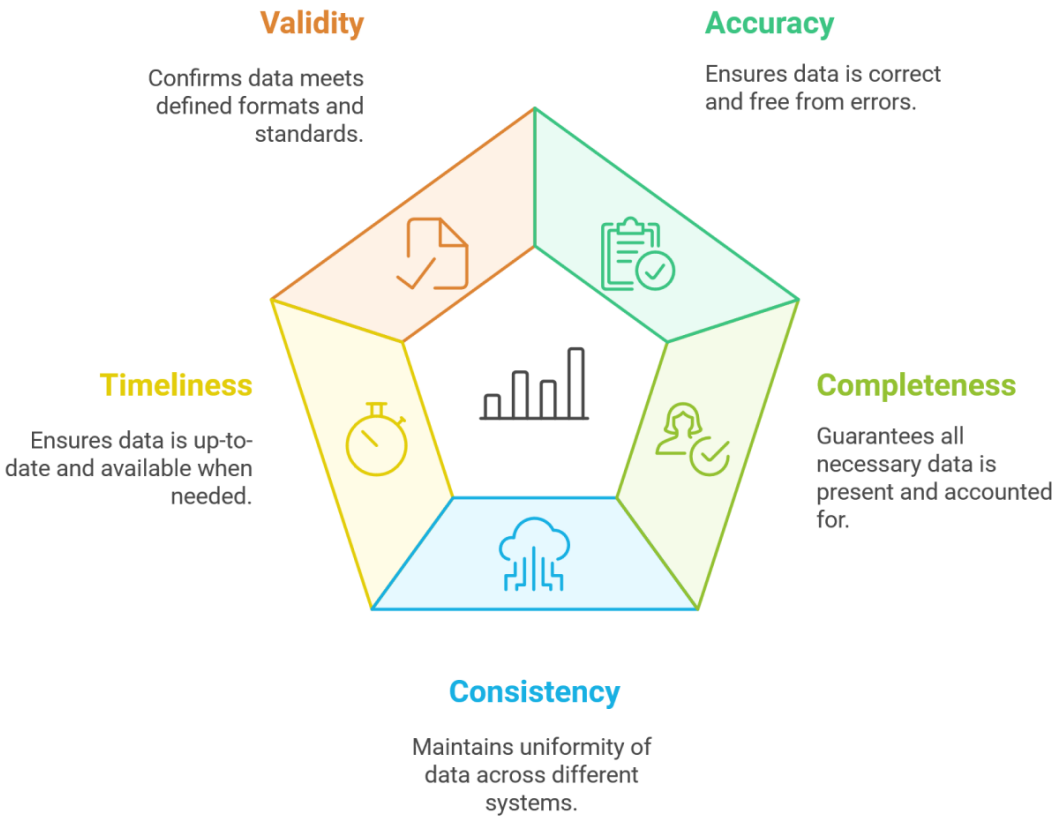
# 2. Asset stories (Lấy dữ liệu bài viết)
@asset
def stories() -> pd.DataFrame:
    """Asset chứa danh sách các bài viết."""
    return pd.DataFrame([
        {"id": 100, "title": "Giới thiệu về Software-Defined Assets trong Dagster"},
        {"id": 101, "title": "Xây dựng Data Pipeline hiện đại"},
    ])

# 3. Asset comment_stories (Kết hợp thông tin từ comments và stories)
@asset
def comment_stories(comments: pd.DataFrame, stories: pd.DataFrame) -> pd.DataFrame:
    """
    Asset kết hợp dữ liệu giữa comments và stories.
    Dagster tự động phát hiện dependency nhờ vào tên tham số 'comments' và 'stories'.
    """
    return pd.merge(
        comments,
        stories,
        left_on="story_id",
        right_on="id",
        how="inner"
    )

# Khai báo Definitions để Dagster UI nhận diện tất cả assets
defs = Definitions(assets=[comments, stories, comment_stories])
```

Data Pipeline

❖ What is Data Quality?



Pillar	What it tracks	What it helps detect
Freshness	Whether data arrives on time	Detects delayed or stalled pipelines before reports break
Volume	Sudden spikes or drops in record counts	Surfaces missing data, duplicates, or upstream ingestion issues
Distribution	Shifts in data values over time	Identifies subtle changes that affect analytical accuracy
Schema	Structural changes in datasets	Catches breaking changes like added, removed, or renamed fields
Lineage	How data flows across systems	Shows where issues originate and how far they propagate

Focuses only on known risks based on fixed, human-defined rules.

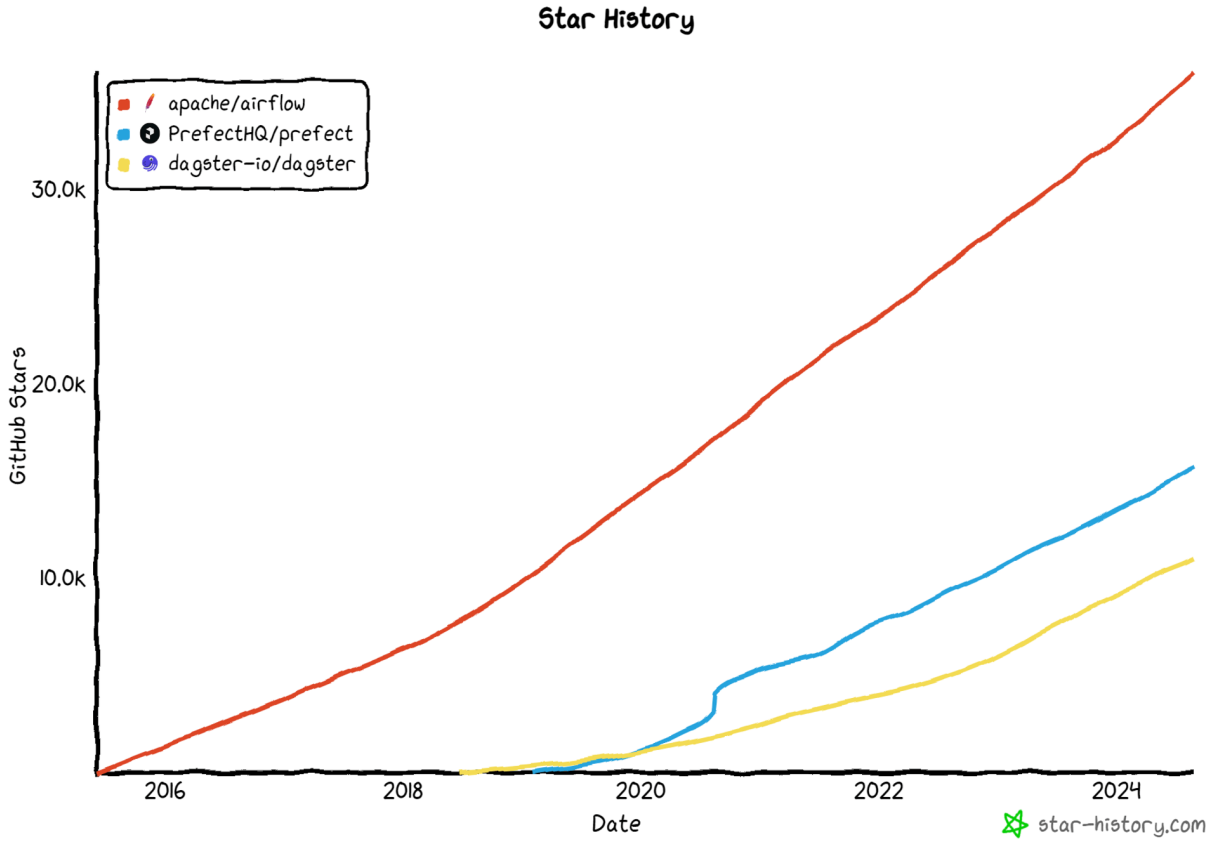
Data Observability

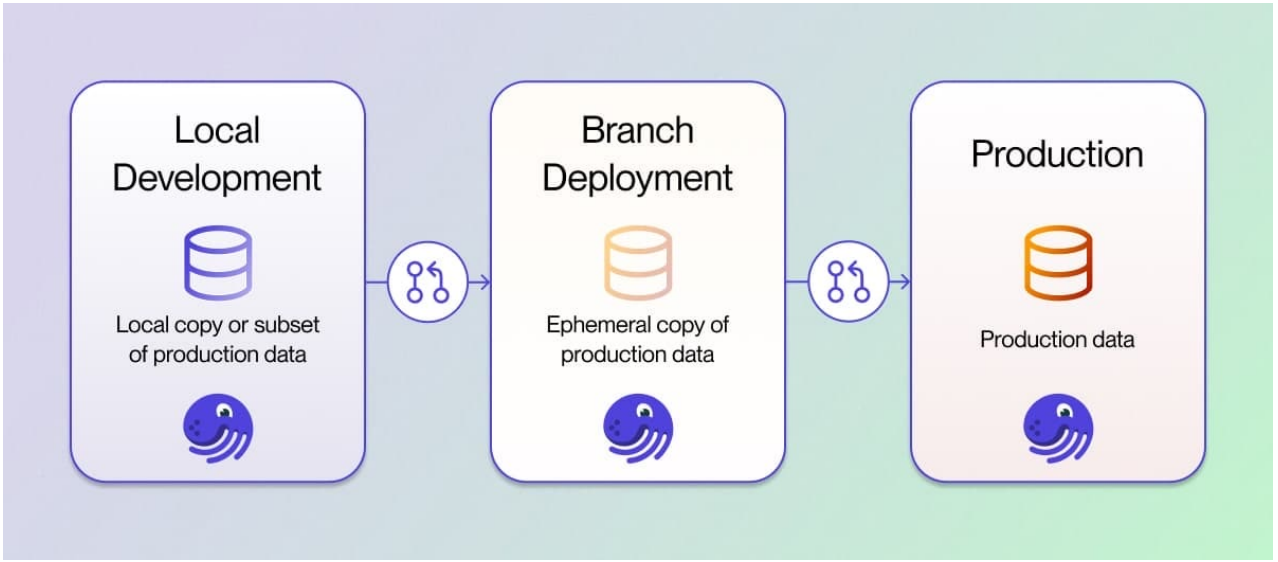
Dagster

Dagster

❖ Getting Started

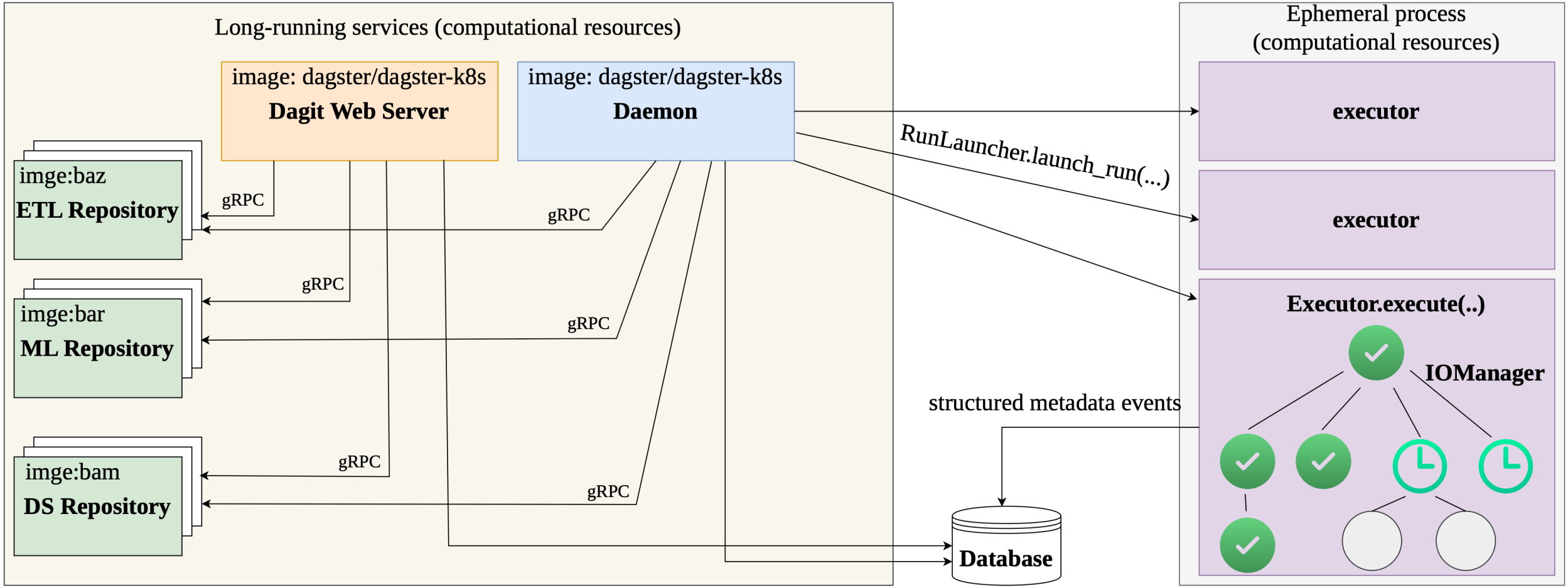
"Dagster is a data orchestrator built for data engineers, with integrated lineage, observability, a declarative programming model, and best-in-class testability."





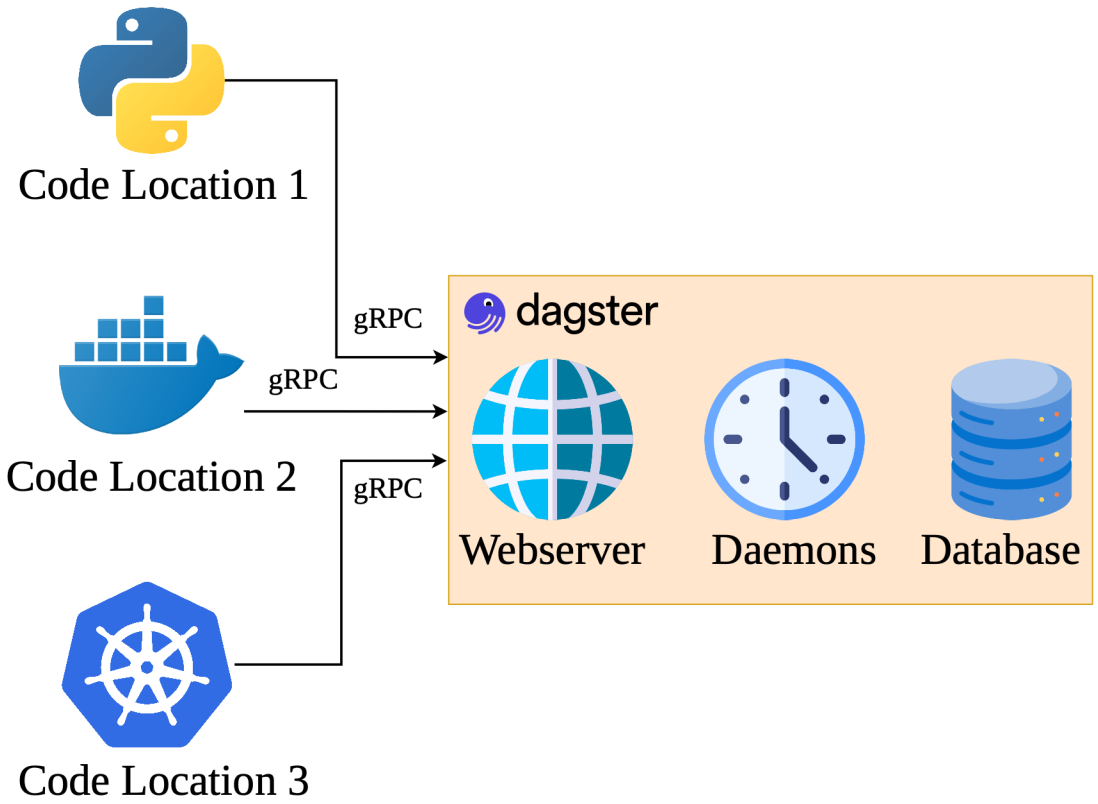
Dagster Architecture

❖ Architecture

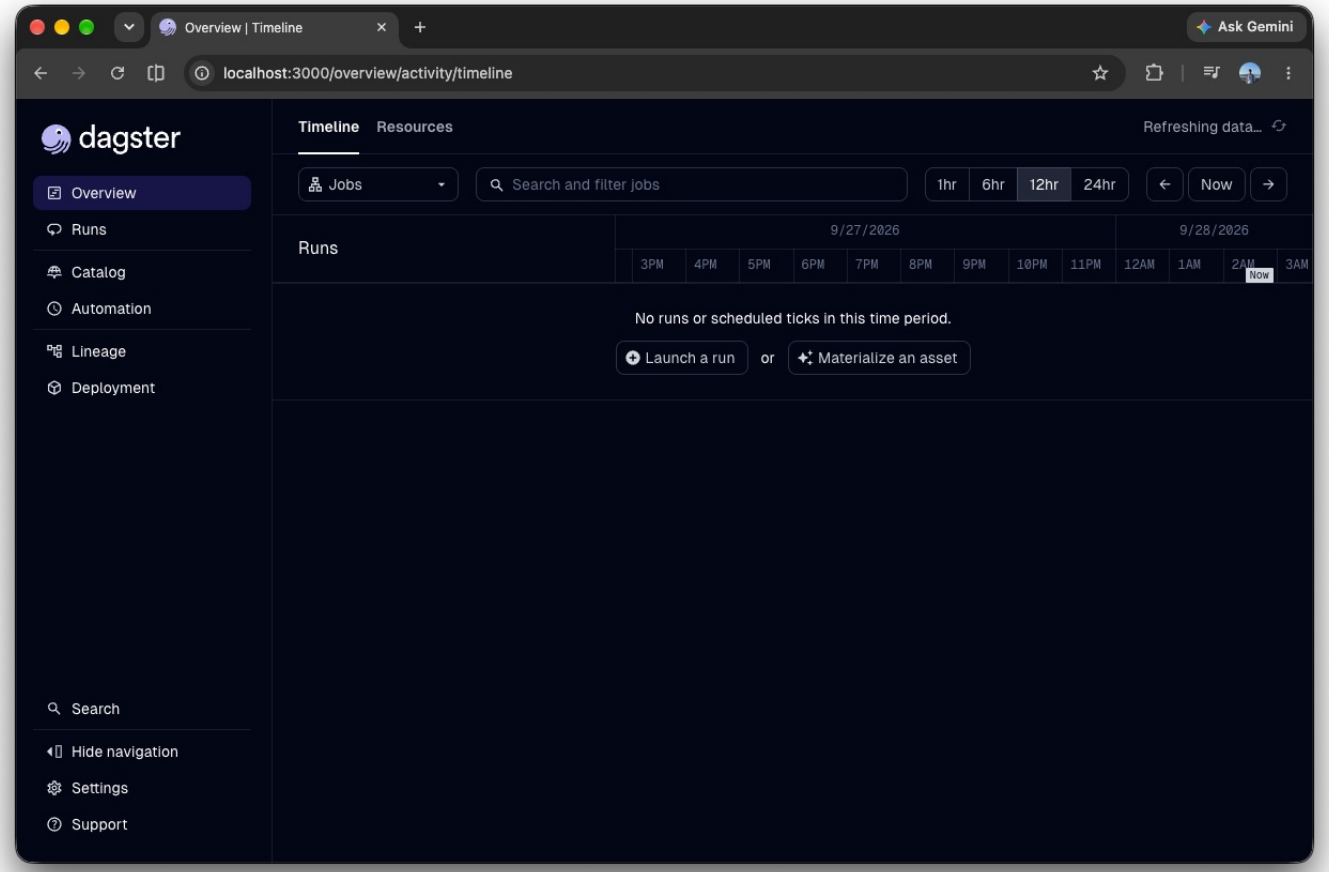


Dagster Architecture

❖ Architecture



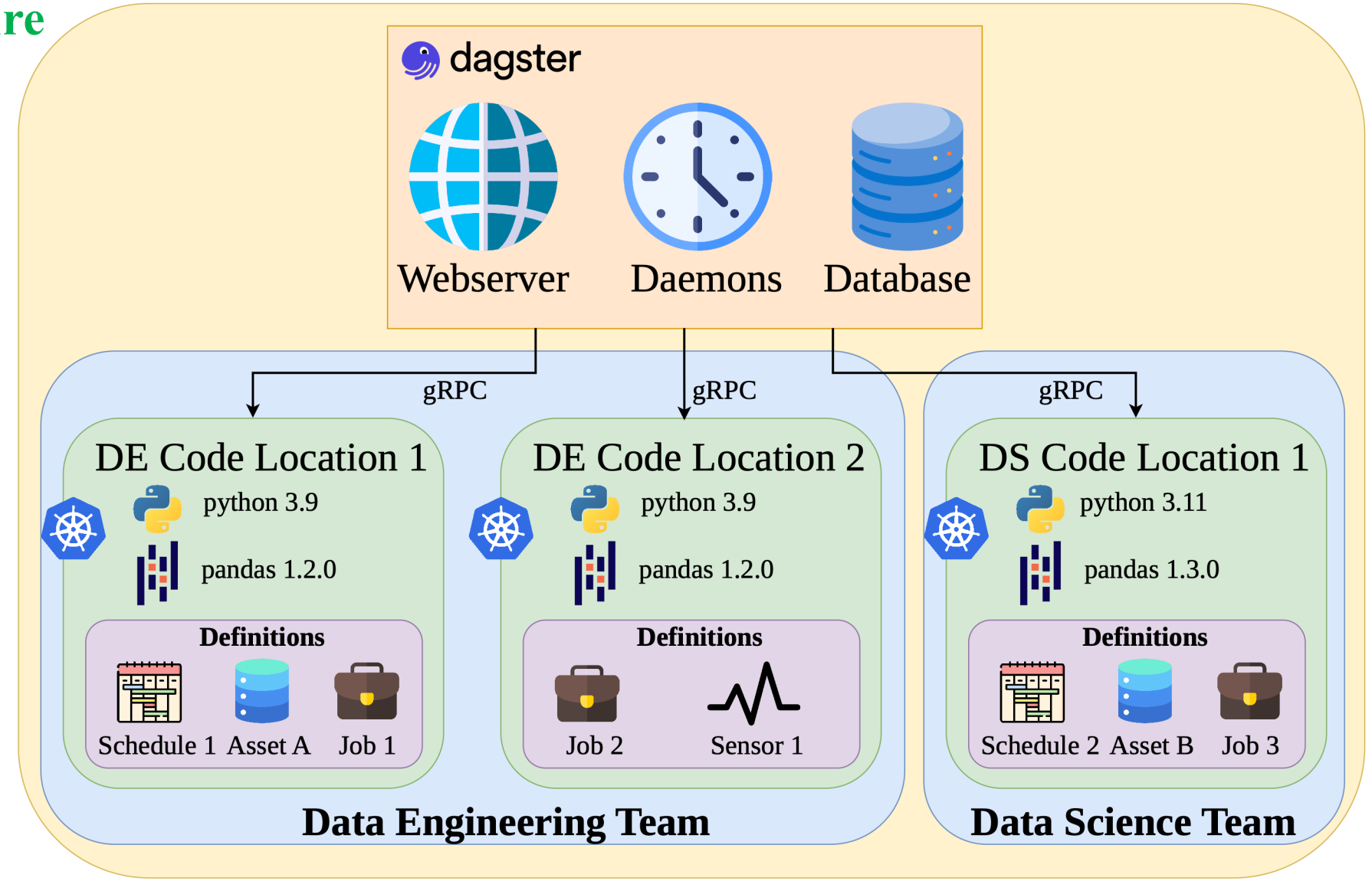
Dagster Webserver



https://docs.dagster.io/deployment/dagster-plus/hybrid/architecture

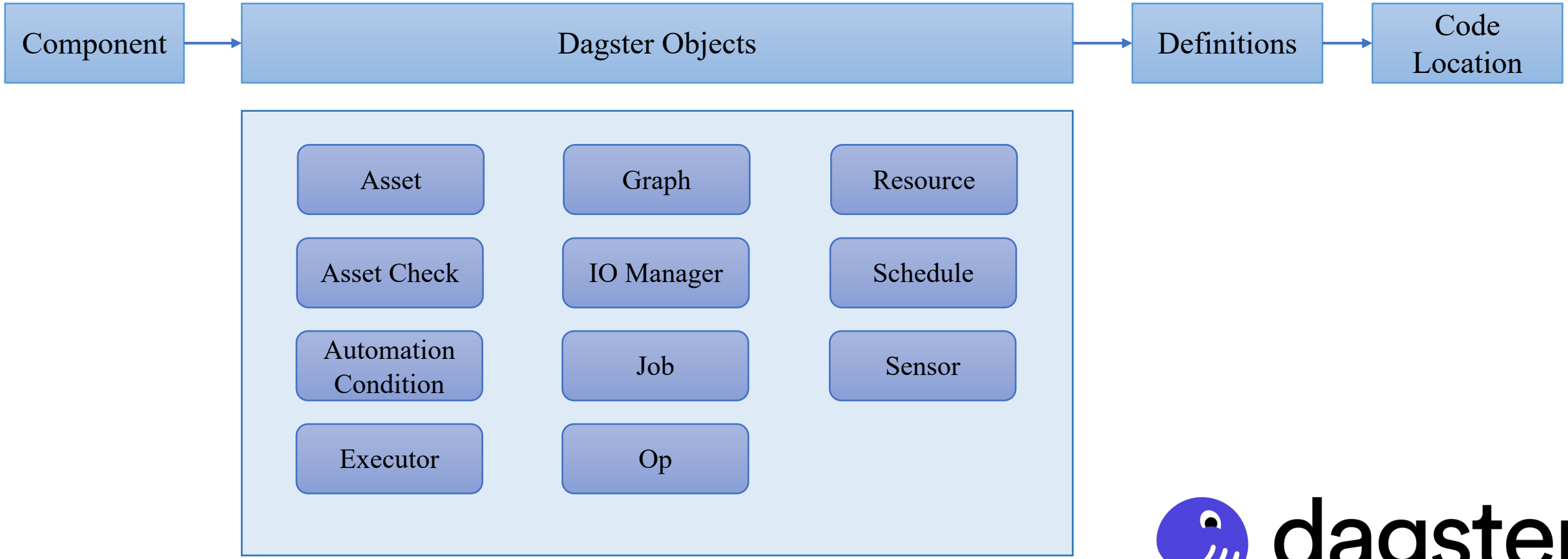
Dagster Architecture

❖ Architecture



Dagster Concepts

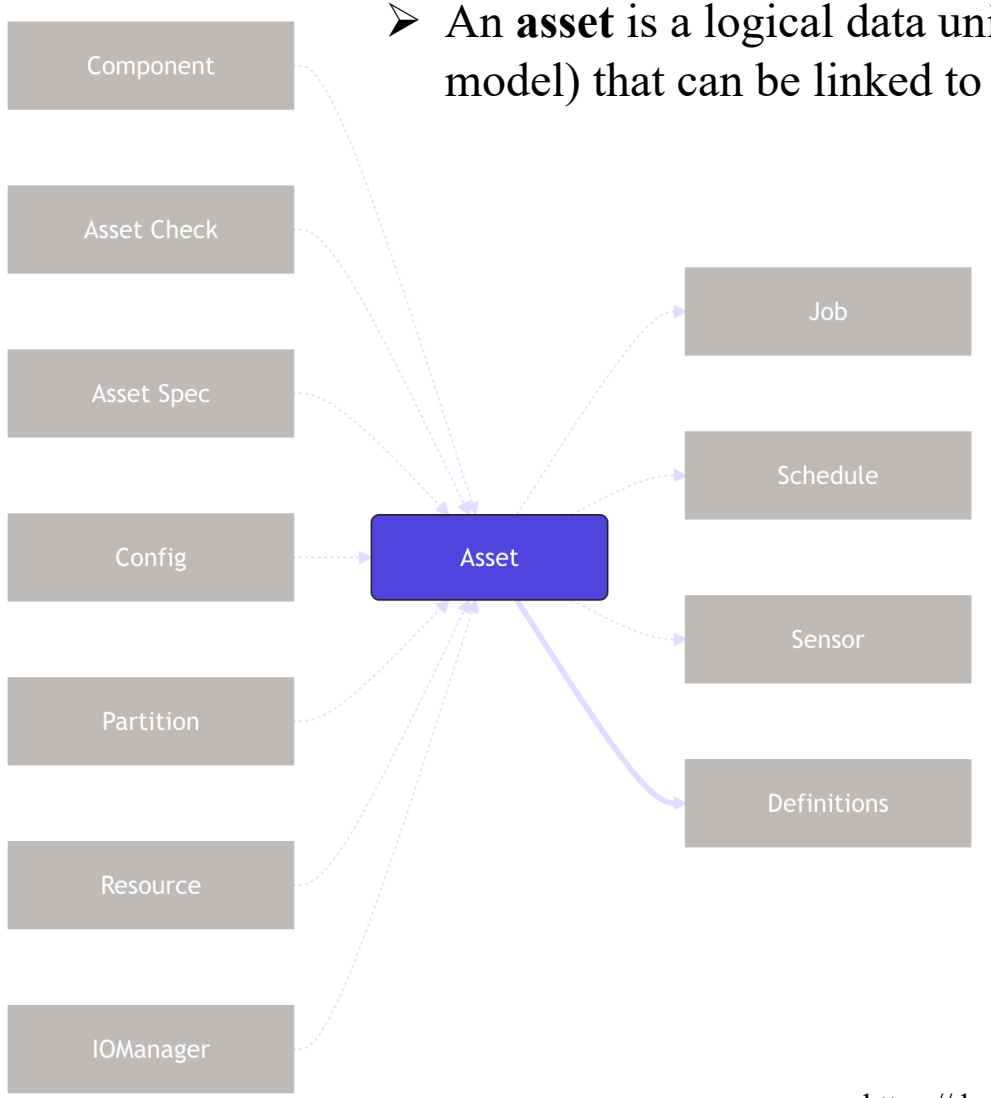
❖ Concepts



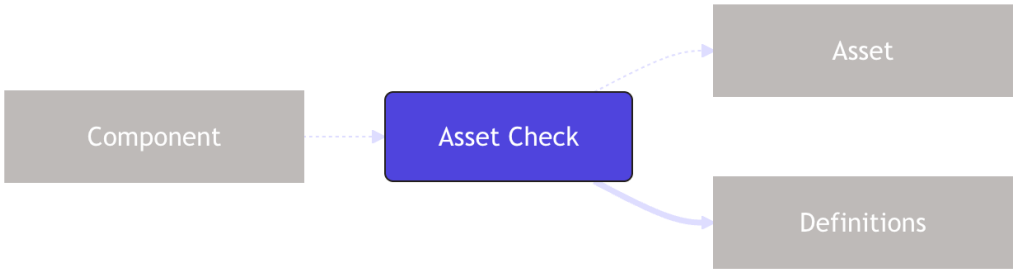
dagster

Dagster Architecture

❖ Asset, Asset check and Specs



- An **asset** is a logical data unit (table, dataset, or model) that can be linked to form **data lineage**.



- An **asset check** verifies an asset's **data quality, freshness, or completeness** when it runs.

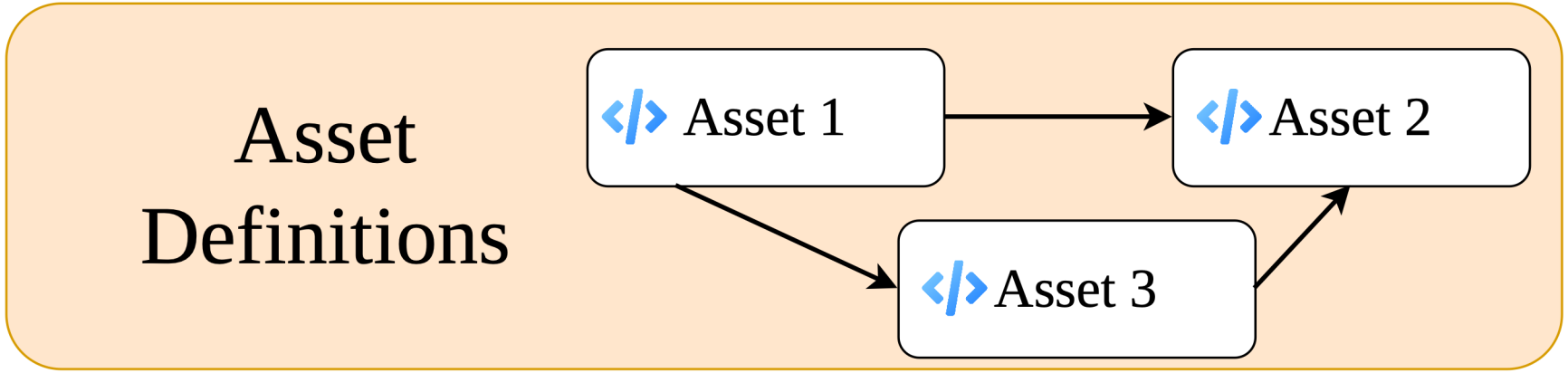
- A **Spec** describes an entity's **identity and metadata** without defining its behavior or execution logic.

https://docs.dagster.io/getting-started/concepts

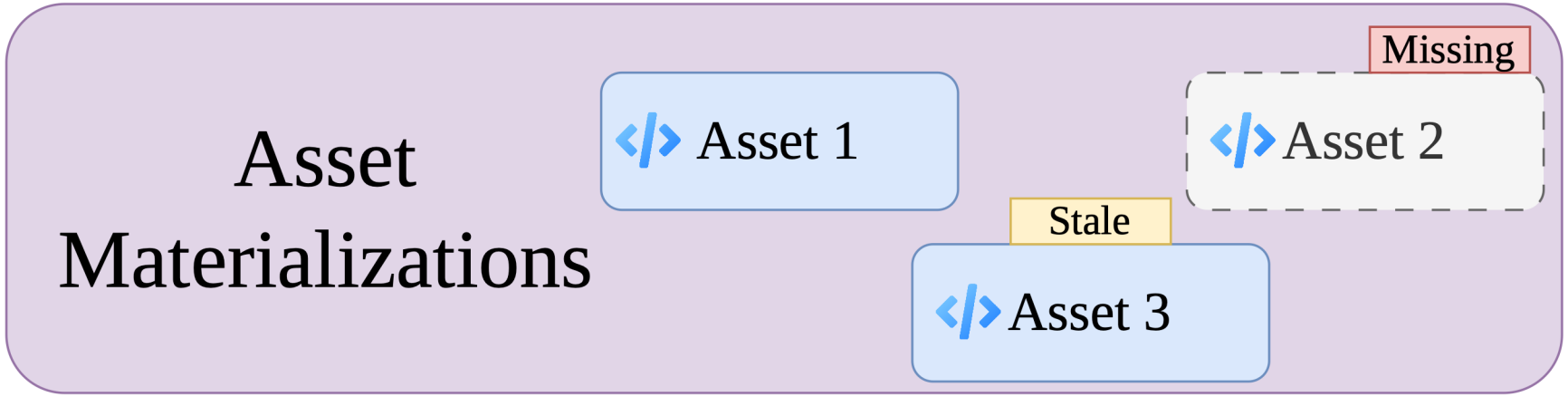
Dagster

❖ Software-Defined Assets (SDA)

- An **asset definition describes assets**, their **dependencies**, and the **logic** required to produce or update it.



- **Asset materialization** is the process of **executing** that code to **compute** the asset and **persist** the results into a **storage**.



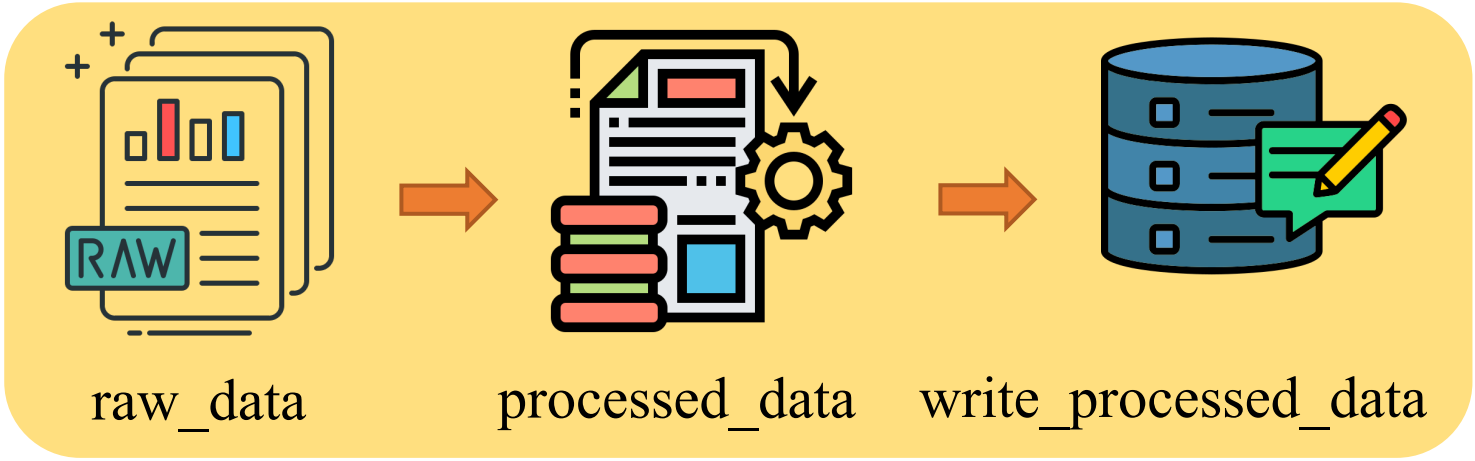
Dagster

❖ Software-Defined Assets (Example)

```python
import pandas as pd
from dagster import asset

# Asset to read data from a CSV file
@asset
def raw_data():
    df = pd.read_csv('input_data.csv')
    return df

# Asset to process the raw data
@asset
def processed_data(raw_data):
    # Imagine some processing logic here, for
example:
    # - Cleaning the data
    # - Filtering rows
    # - Transforming columns
    # For simplicity, we'll just add a new
column with transformed data
    processed_df = raw_data.copy()
    processed_df['new_column'] =
raw_data['existing_column'] * 2
    return processed_df
```



1. raw_data: **Reads** data from an input CSV.
2. processed_data: Takes the raw data DataFrame as input, performs some and returns the **processed** DataFrame.
3. write_processed_data: Takes the processed DataFrame and **writes** it to a new CSV file named processed_data.csv.

```python
@asset
def write_processed_data(processed_data):
    processed_data.to_csv('processed_data.csv', index=False)
```

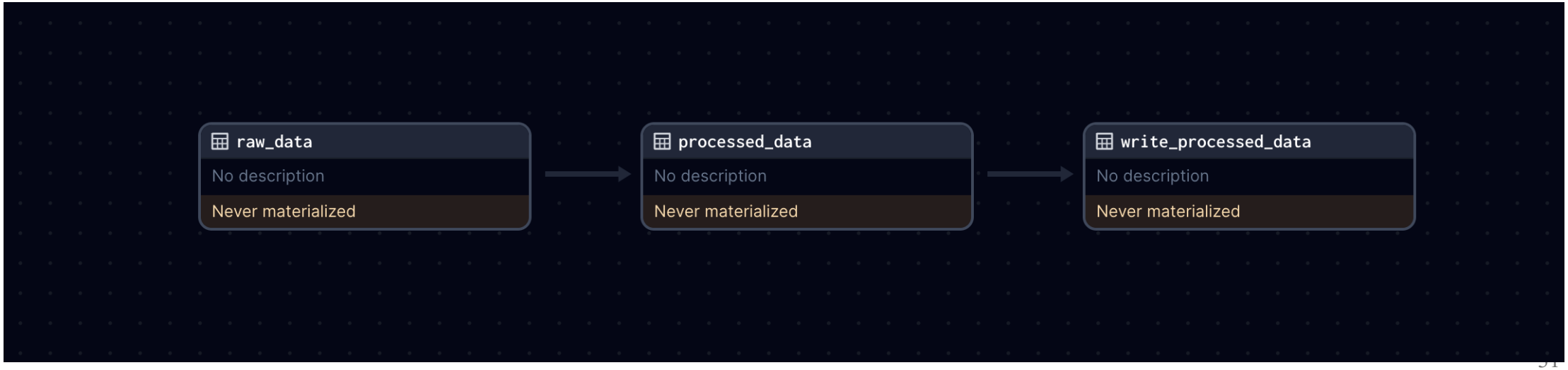
Dagster

❖ Software-Defined Assets (Example)

- To make these assets discoverable by Dagster, you would make sure they are imported in your repository defs:

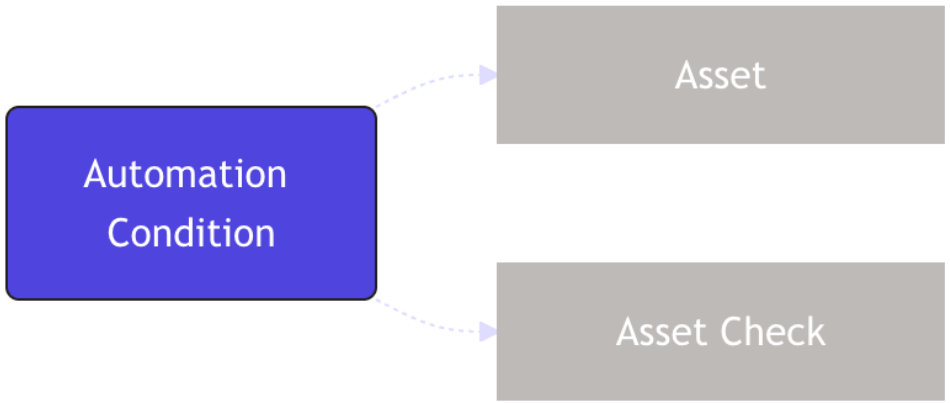
```python
# __init__.py
from dagster import Definitions, load_assets_from_modules
from . import assets
all_assets = load_assets_from_modules([assets])
defs = Definitions(
    assets=all_assets,
)
```

Result (Dagster UI):



Dagster Architecture

❖ Automation condition & Code location





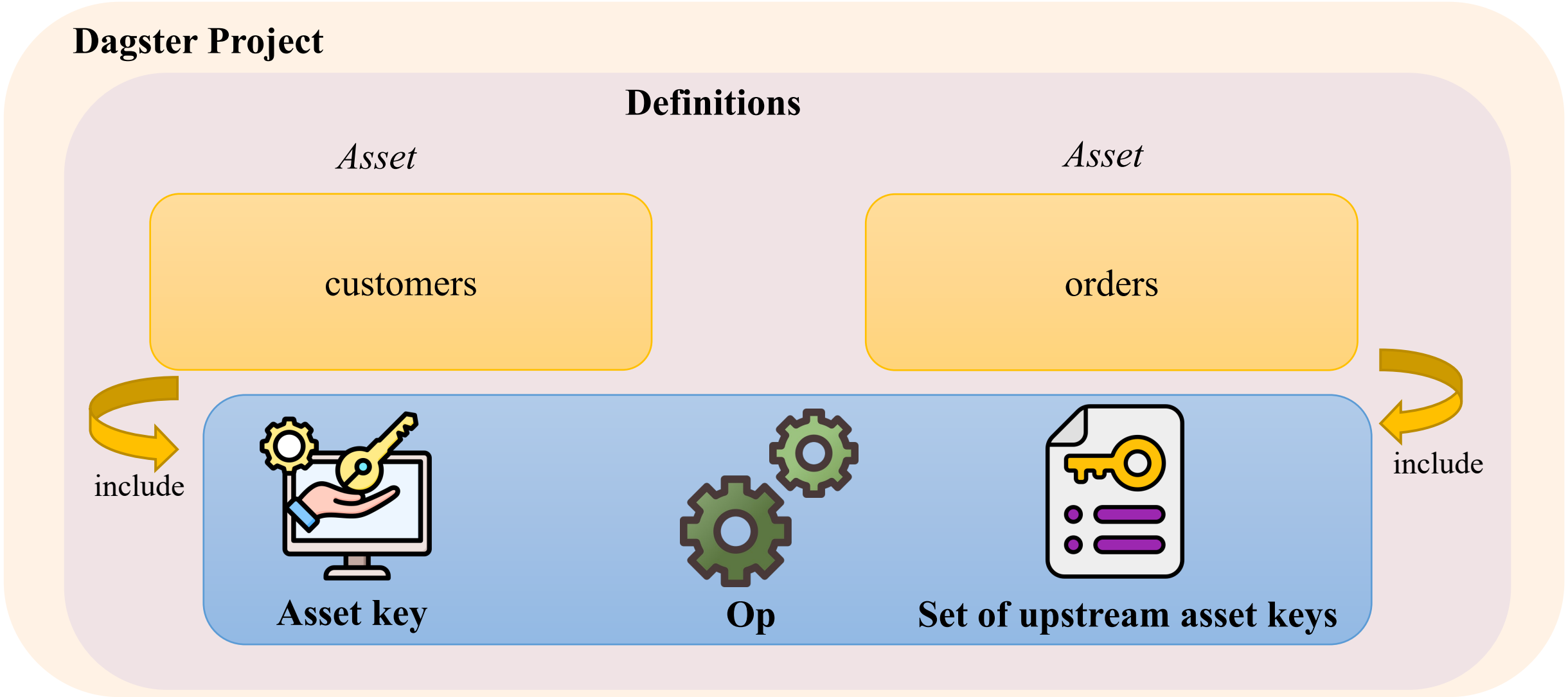
- An AutomationCondition defines when an asset or asset check should run based on its status and upstream dependencies.

- A **code location** is a collection of Dagster definitions deployed in a specific **Python environment**, with its own dependencies.

https://docs.dagster.io/getting-started/concepts

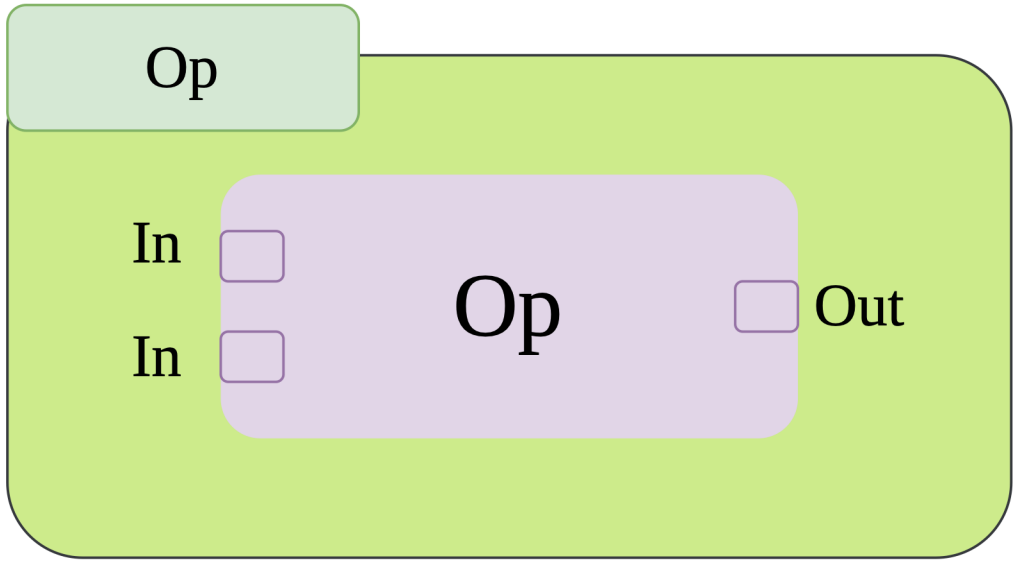
Dagster

❖ Software-Defined Assets (SDA)



Dagster

❖ Ops




```python
import dagster as dg
class MyOpConfig(dg.Config):
    api_endpoint: str

@dg.op
def my_configurable_op(config: MyOpConfig):
    data =
requests.get(f"{config.api_endpoint}/data").jso
n()
    return data
```



- **Ops** are the **core unit of computation** in Dagster.
- The **computational core** of an **asset** definition is **an op**. Collections of ops can also be assembled to create a graph.

An individual op should perform relatively simple tasks



Deriving a dataset from other datasets



Executing a database query



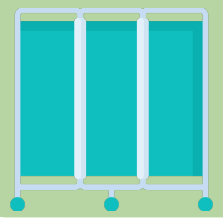
Initiating a Spark job in a remote cluster



Querying an API and storing the result in a data warehouse

Dagster

❖ Partitions



➢ In Dagster, **partitioning** is a powerful **technique** for **managing large datasets**, **improving pipeline performance**, and **enabling incremental processing.**

Static Partitions

per_customer_features

- coca_cola_co
- pepsi_co
- fanta_co

Time-based partitions

events_table

- 2026-09-09
- 2026-09-10
- 2026-09-11

Multi Partitions

per_customer_features

	pepsi	coca_cola
2026-09-09		
2026-09-10		
2026-09-11		

Several ways to partition:

1. **Time-based partitioning**: for processing data in specific time intervals.
2. **Static partitioning**: for dividing data based on predefined categories.
3. **Two-dimensional partitioning**: for partitioning data along two different axes simultaneously.
4. **Dynamic partitioning**: for creating partitions based on runtime information.

```python
from dagster import DailyPartitionsDefinition, HourlyPartitionsDefinition, asset

@asset(partitions_def=HourlyPartitionsDefinition(start_date="2026-01-01-00:00"))
def events():
    ...
@asset(partitions_def=HourlyPartitionsDefinition(start_date="2026-01-01-00:00"))
def logins(events):
    ...
@asset(partitions_def=DailyPartitionsDefinition(start_date="2026-01-01"))
def logins_cleaned(events, logins):
    ...
```

Dagster

❖ Schedules



- **Schedules** enable **automated** execution of **jobs** at **specified intervals**.
- Common **frequencies** like **hourly, daily, or weekly**, to more complex **patterns** defined using **cron expressions**.

```
# * * * * *  command to execute
# ┬ ┬ ┬ ┬ ┬
# │ │ │ │ │
# │ │ │ │ └──── day of week (0 - 7)
# │ │ │ └─────── month (1 - 12)
# │ │ └──────── day of month (1 - 31)
# │ └───────── hour (0 - 23)
# └──────── min (0 - 59)
```

```python
import dagster as dg

@dg.asset
def customer_data(): ...

@dg.asset
def sales_report(): ...

daily_schedule = dg.ScheduleDefinition(
    name="daily_refresh",
    cron_schedule="0 0 * * *",
    execution_timezone="America/Los_Angeles",
    target=[customer_data, sales_report],
)


defs = dg.Definitions(schedules=[daily_schedule])
```

- Runs at midnight daily
- Use the timezone: “America/Los_Angeles”

Dagster

❖ Sensors



- **Sensors** enable you to **take action in response** to **events** that **occur** either internally **within Dagster** or **in external systems**.
- **Sensors** check for events at **regular intervals** and **either perform an action**.

Example of events



A run completes in Dagster



A run fails in Dagster



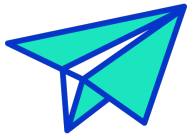
A file appears in an s3 bucket



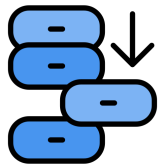
Example of actions



Launching a run



Sending a Slack message



Inserting a row into a database



Dagster

❖ Sensors

```python
import dagster as dg

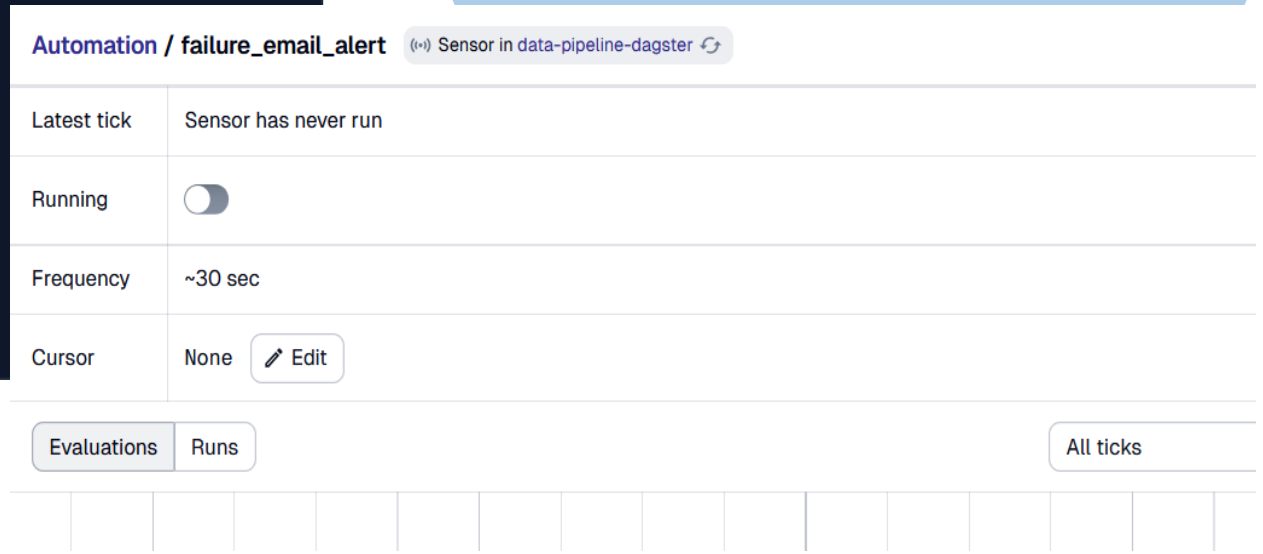
def email_subject(context: dg.RunFailureSensorContext) -> str: ...
def email_body(context: dg.RunFailureSensorContext) -> str: ...  # HTML: job, run_id, errors each step, link UI

failure_email_sensor = dg.make_email_on_run_failure_sensor(
    name="failure_email_alert",
    email_from="me@gmail.com",
    email_password="gmail-app-password",
    email_to=["alert@example.com"],
    email_subject_fn=email_subject,
    email_body_fn=email_body,
    monitor_all_code_locations=True,
    default_status=dg.DefaultSensorStatus.RUNNING,
)

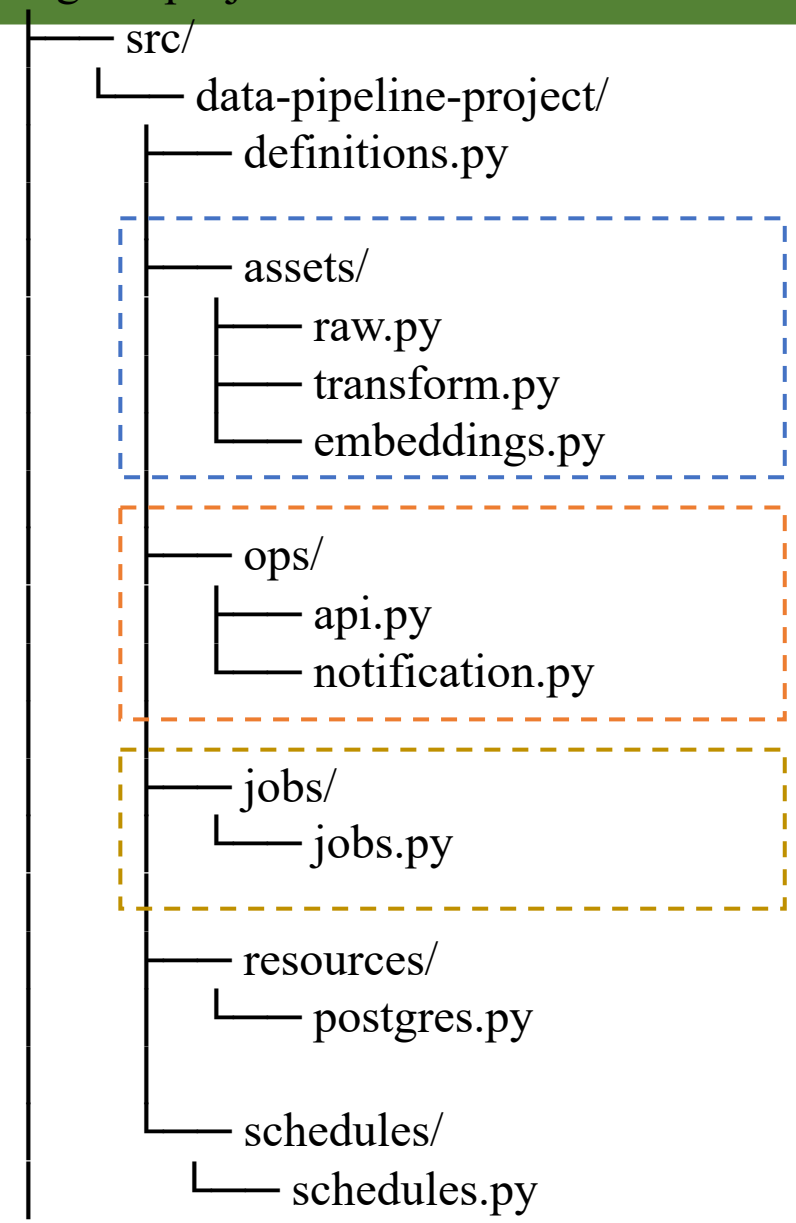
defs = dg.Definitions(sensors=[failure_email_sensor])
```

Flow

1. any run FAILURE
2. daemon calls sensor
3. email_subject() + email_body()
4. send via Gmail SMTP
5. update cursor



Dagster



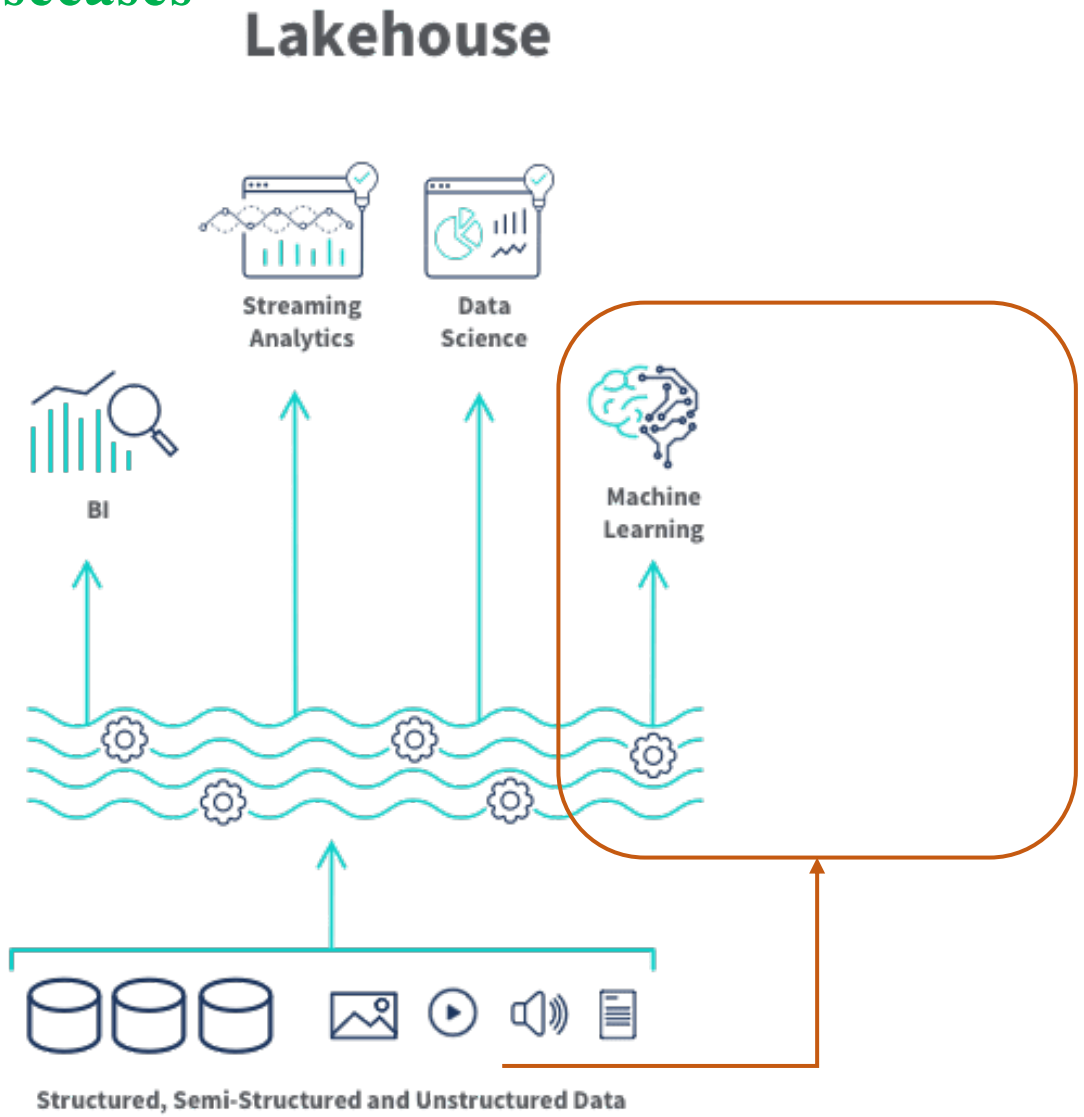
Use this when you are interested in **generating data**.

Use this when you are concerned about the **task** that needs to be run.

Use this when you want to run a **workflow**.

Dagster

❖ Usecases



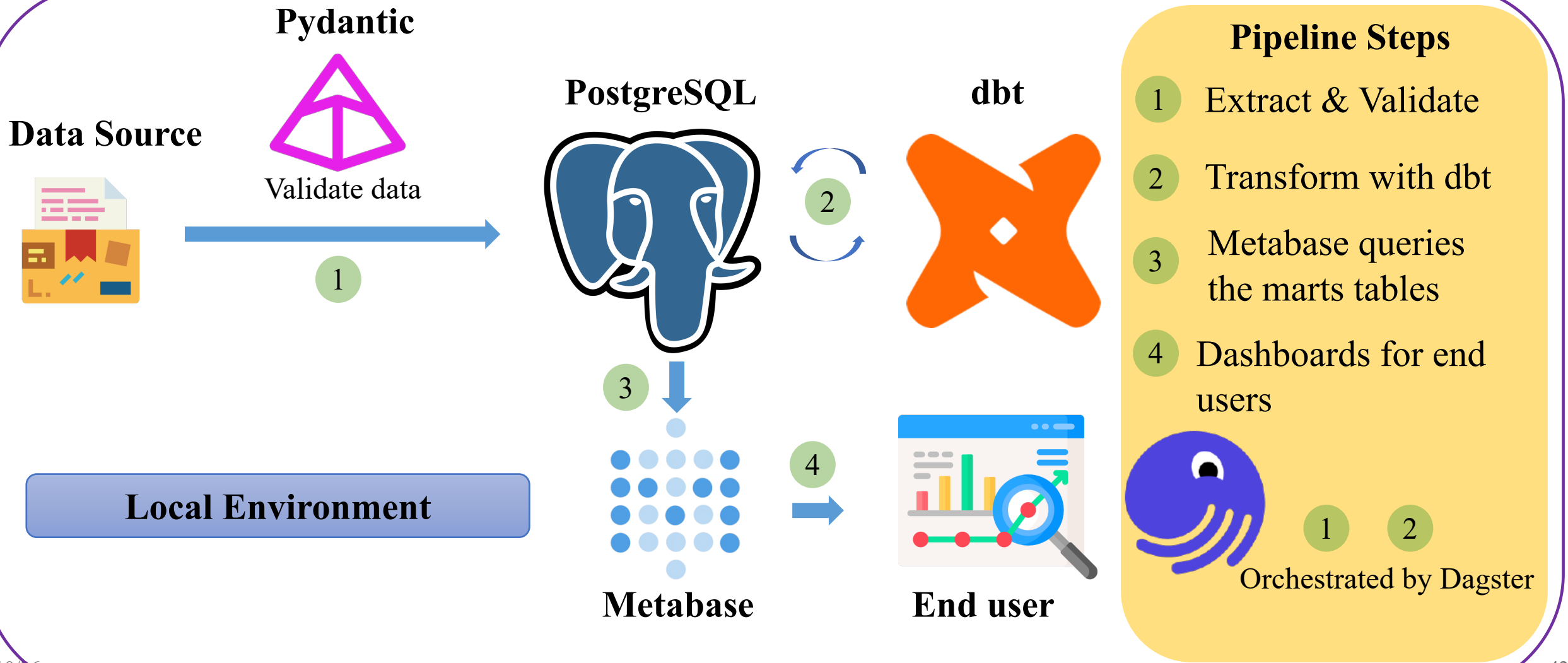
- Machine Learning
- Deep Learning
- LLMs
- Agents



Build with Dagster

Deployment

❖ Overview Architecture

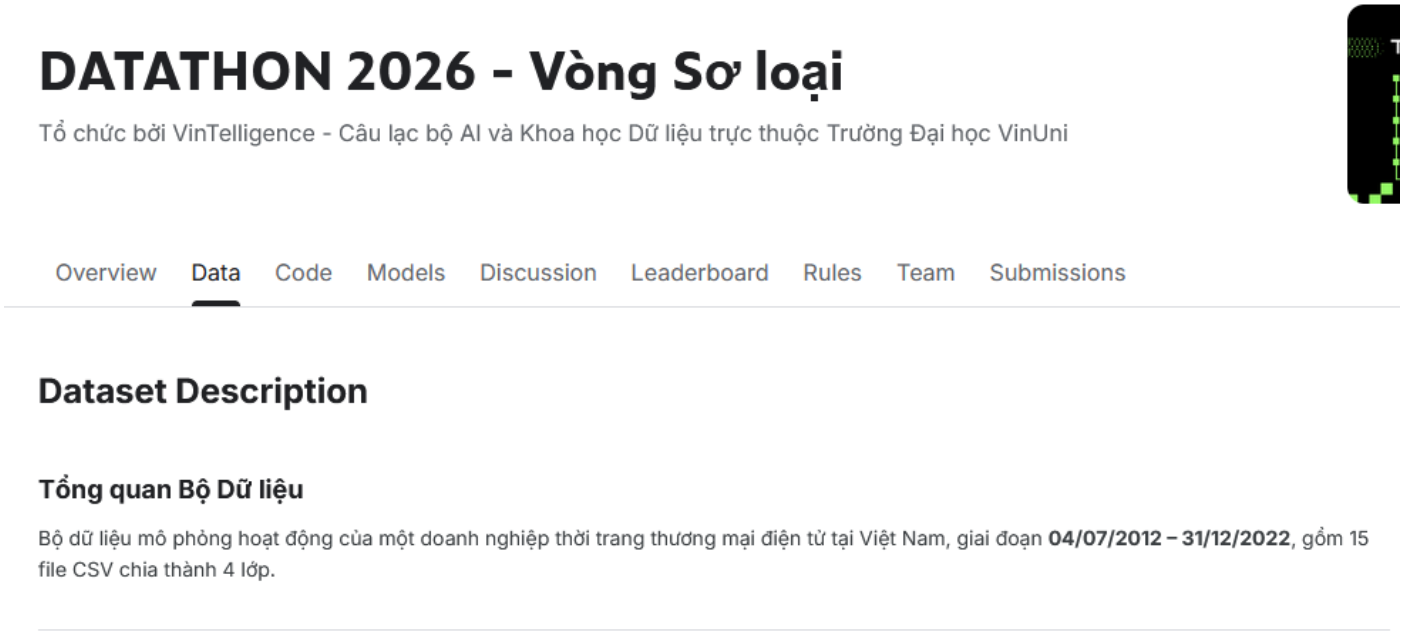


Deployment

❖ Getting Started

Vindatathon 2026 Round 1





https://www.kaggle.com/competitions/datathon-2026-round-1/data

Nhìn nhanh 6 bảng raw, key chính và quan hệ 1:N.

Deployment

❖ Ingestion layer

Previous Project

Input

CSV in folder

- customers.csv
- order_items.csv
- orders.csv
- products.csv
- payments.csv
- payments.csv
- reviews.csv

Each CSV file will become a corresponding raw table.

Operation

Commands used in the slides

```
python -m data_pipeline.ingest
Completed successfully
```

- **Reset raw schema**
- Read each CSV file
- Validate every row

Reject now
Ingest_errors.csv

Output

Tables are created (raw)

- raw_customers
- raw_orders
- raw_order_items
- raw_products
- raw_payments
- raw_reviews

Deployment

❖ Ingestion layer

Previous Project

Operation Commands used in the slides

```
python -m data_pipeline.ingest
Completed successfully
```

- **Reset raw schema**
- Read each CSV file
- Validate every row

Reject now Ingest_errors.csv



- Full reload every run

```
DROP SCHEMA IF EXISTS {DB_SCHEMA} CASCADE
```

Daily partitions: pre-split files, backfill any day

- Error log overwritten every run

```
errors_file = open(errors_path, "w")   # "w" = overwrite
```

Errors tracked per partition: row, field, and reason, kept for every run

Deployment

❖ Ingestion layer

Static Partitions

per_customer_features

- coca_cola_co
- pepsi_co
- fanta_co

Time-based partitions

events_table

- 2026-09-09
- 2026-09-10
- 2026-09-11

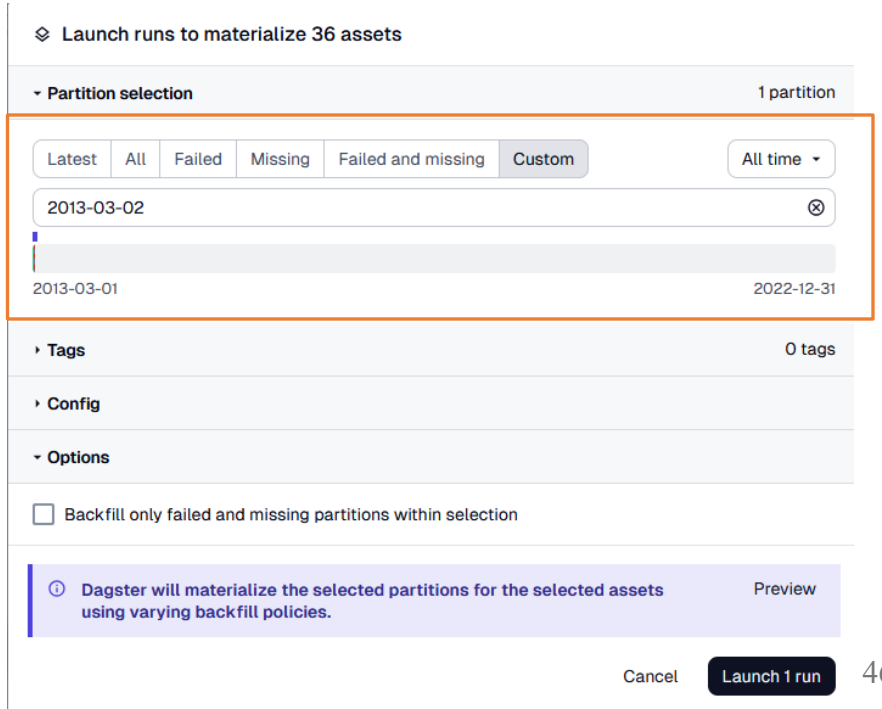
Multi Partitions

per_customer_features

	pepsi	coca_cola
2026-09-09		
2026-09-10		
2026-09-11		

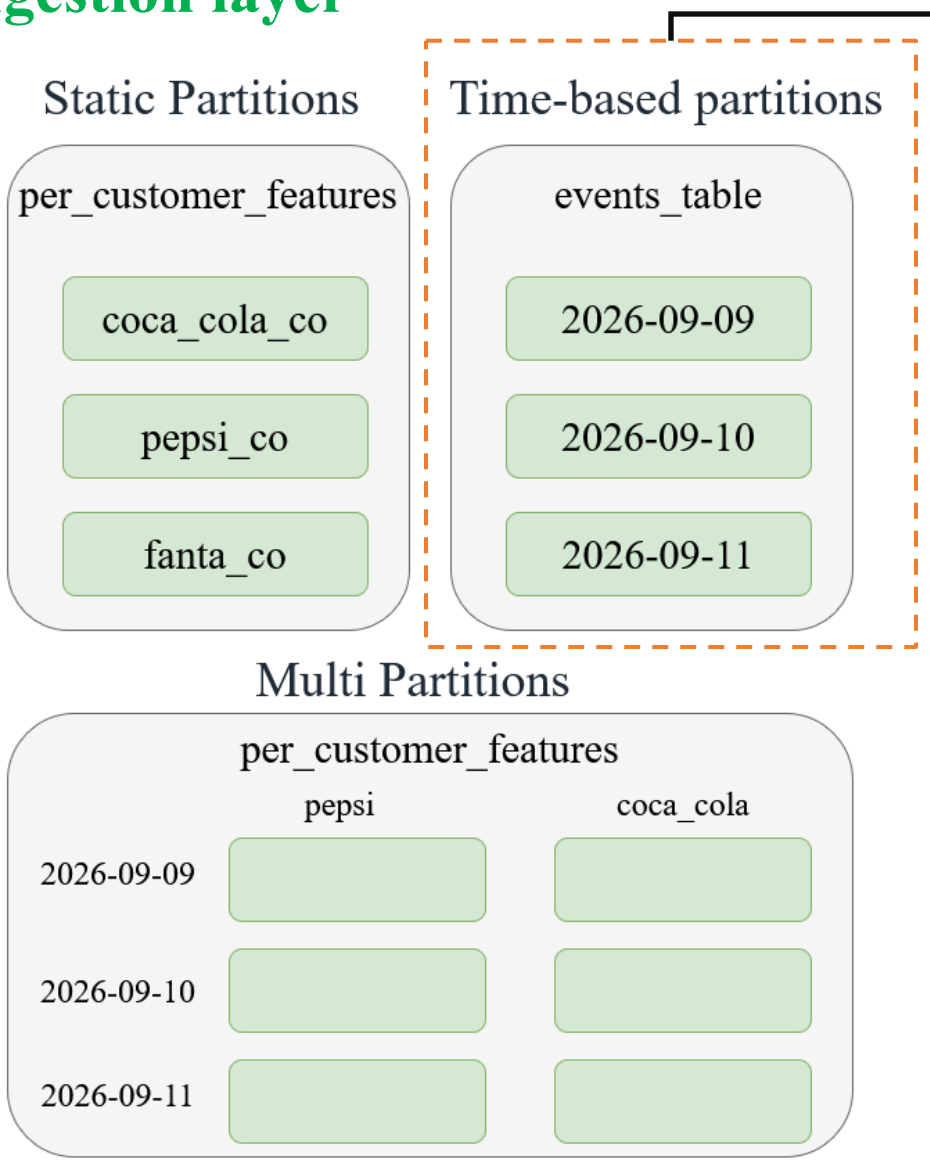
Time-based partitioning, for processing data in specific time intervals.





Deployment

❖ Ingestion layer



Time-based partitioning, for processing data in specific time intervals.

Schedules from partitions

The **schedule** will **execute** at the **same cadence** specified by the **partition definition**.

```python
daily_pipeline_job = dg.define_asset_job(
    name="daily_pipeline_job",
    selection=dg.AssetSelection.groups("ingestion",
"ingestion_dimensions").downstream(include_self=True),
    partitions_def=daily_partitions,
)
```

Deployment

❖ Ingestion layer



It **simulates** how **data arrives** in a production system: **a little bit every day, not all at once**.

dagster

- Overview
- Runs
- Catalog
- Jobs
- Automation
- Lineage
- Deployment

Automation / daily_pipeline_job_schedule — Schedule in data-pipeline-dagster

Latest tick	Schedule has never run
Next tick	Sep 25, 12:00:00 AM UTC
Target	daily_pipeline_job
Running	
Partition set	None
Schedule	At 12:00 AM UTC `(0 0 * * *)`
Execution timezone	UTC

Deployment

❖ Ingestion layer

Previous Project

- Error log overwritten every run
- Example samples in ingest_errors.csv:

	A	B	C	D	E	F	G	H	I	J
1	table,row_number,error									
2	customers,23,"1 validation error for CustomerRow									
3	gender									
4	Input should be 'Female', 'Male' or 'Other' [type=enum, input_value='Non-binary', input_type=str]									
5	For further information visit https://errors.pydantic.dev/2.12/v/enum"									
6	customers,36,"1 validation error for CustomerRow									
7	gender									
8	Input should be 'Female', 'Male' or 'Other' [type=enum, input_value='Non-binary', input_type=str]									
9	For further information visit https://errors.pydantic.dev/2.12/v/enum"									
10	customers,82,"1 validation error for CustomerRow									
11	gender									
12	Input should be 'Female', 'Male' or 'Other' [type=enum, input_value='Non-binary', input_type=str]									
13	For further information visit https://errors.pydantic.dev/2.12/v/enum"									
14	customers,86,"1 validation error for CustomerRow									



Current Project

Metadata per run: every asset run reports row counts, reject rate, and a table of rejected rows to Dagster, stored per table and per partition

```
def _asset(context, postgres: PostgresResource) ->
dg.MaterializeResult:
    partition_date = context.partition_key
    stats = ingest_daily_partition(table_name,
postgres.get_engine(), partition_date)

    return dg.MaterializeResult(metadata={
        "rows_read": stats["rows_read"],
        "reject_rate":
dg.MetadataValue.float(round(stats["reject_rate"], 4)),
        "rejected_row_samples":
dg.MetadataValue.md(error_samples_markdown(stats["sample_e
rrors"])),
    })
```

Deployment

❖ Data Ingestion

Assets › raw › raw_customers

Overview | Partitions | **Events** | Checks | Lineage

Filter

- Sep 24, 6:11 PM — 2013-03-10
- Sep 24, 6:10 PM — 2013-03-10
- Sep 23, 9:23 PM — 2013-03-02
- Sep 23, 2:31 PM — 2013-03-02
- Sep 23, 2:19 PM — 2013-03-01
- Sep 23, 2:17 PM — 2013-03-01

Sep 24, 6:10 PM

Partition 2013-03-10 materialized in run fdbb43ef

Metadata

partition_date	2013-03-10
rows_read	8
rows_valid	7
rows_rejected	1
reject_rate	0.125
rejected_row_samples	row: 8 · field: gender · type: enum · message: Input should be 'Female', 'Male' or 'Non-binary'

Test: insert a row ‘gender=unknown’

Tags

code_version	fdbb43ef-2262-4e3c-a11c-03f569d6fd9a
data_version	27f383176d7a71fa8626c6fb8ebd9832831fb4bcf0c6480a3d9042025e6e2071

Hide tags

Deployment

❖ Data Ingestion

```python
def ingest_daily_partition(table_name, engine,
partition_date):
    csv_path = DAILY_DIR / table_name /
f"{partition_date}.csv"
    ...
    if date_col is not None:
        delete_clause = f"WHERE {date_col} = :d"
    else:
        delete_clause = "WHERE order_id = ANY(:ids)"
```

```python
def ingest_dimension_table(table_name, engine):
    csv_path = DATASET_DIR / f"{table_name}.csv"
    ...
    _replace_rows(..., "WHERE TRUE", {})
```

Daily table have a **date**, so we **load** them **day by day**. **Static table** have **no daily grain** so we **reload them in full.**

Daily tables

- customers: signup_date
- orders: order_date
- shipments: ship_date
- returns: return_date
- reviews: review_date
- web_traffic: date
- sales: date
- order_items
- payments

Static tables

- product
- geography
- promotion
- inventory

➢ order_items and payments borrow the parent order's date.

Deployment

❖ Transform with dbt & Dagster

Previous Project

- **olist_dbt_practice/** — project root
  - **seeds/** — raw_*.csv
  - **models/**
    - **staging/** — sources.yml, schema.yml, stg_*.sql
    - **intermediate/** — schema.yml, int_*.sql
    - **marts/** — schema.yml, mart_*.sql
  - **macros/** — generate_schema_name.sql
  - **tests/** — singular SQL tests
  - **expected_outputs/**
  - **dbt_project.yml**
    profiles.yml.example, requirements.txt, README.md, scripts/

1 sources → 2 staging → 3 intermediate → 4 marts



Current Project

Remain 23 models

The **dagster-dbt library** provides a **DbtProjectComponent,** which can be **used** to easily **represent dbt models as assets** in Dagster.

The usual Dagster + dbt setup is one YAML file (defs.yaml)

```
type: dagster_dbt.DbtProjectComponent

attributes:
  project: '{{ context.project_root }}/dbt_project'
  exclude: 'int_daily_metrics int_return_analysis
mart_daily_sales_features'
```



https://docs.dagster.io/integrations/libraries/dbt

Deployment

❖ Transform with dbt & Dagster

Previous Project: int_daily_metrics.sql

```
with daily_orders as (
    select
        order_date as date,
        count(distinct order_id) as order_count,
        count(distinct customer_id) as
unique_customers
    from {{ ref('stg_orders') }}

    group by 1
),
-- daily_sales, daily_traffic ...
```

The **model requires** the **date** of the running **partition**.

```
run_date = context.partition_key
# example "2013-03-05", not date.today()
```

Current Project: int_daily_metrics.sql

```
{{ config(
    materialized='incremental',
    incremental_strategy='delete+insert',
    unique_key='date'
) }}

with daily_orders as (
    select
        order_date as date,
        count(distinct order_id) as order_count,
        count(distinct customer_id) as
unique_customers
    from {{ ref('stg_orders') }}
    {% if is_incremental() %}
    where order_date = '{{ var("run_date") }}'::date
    {% endif %}
    group by 1
),
-- daily_sales, daily_traffic: same filter
```

Deployment

❖ Transform with dbt & Dagster

Current Project: dbt_incremental.py

```
def incremental_rolling_window_assets(context, dbt):
    run_date = context.partition_key                          # "2013-03-05"
    yield from dbt.cli(
        ["build", "--vars", json.dumps({"run_date": run_date})],
        context=context,
    ).stream()
```



SQL filters on `var("run_date")`, which must come from the Dagster partition

Command

```
dbt build --select int_daily_metrics int_return_analysis
mart_daily_sales_features \
          --vars '{"run_date": "2013-03-05"}'
```

```
where order_date = '{{ var("run_date") }}'::date
# where order_date = '2013-03-05'::date
```

Current Project: int_daily_metrics.sql

'context.partition_key' → 'dbt build --vars '{"run_date": ...}': the **partition date** becomes the **dbt variable**.

https://docs.dagster.io/integrations/libraries/dbt

Deployment

❖ Quality gates

Add Dagster Asset check

```
@dg.asset_check(asset=asset_def,
name=f"{table_name}_row_count_positive",
blocking=True)


@dg.asset_check(asset=asset_def,
name=f"{table_name}_reject_rate_below_threshold")
```

Pydantic



Quality Gate 1

Row validation

- Data types
  int, date, Decimal
- Allowed values
  Enums, gt / ge / le ranges
- Decimal precision
  15 digits, 2 decimal places
- Cross-field rules
  ship_date ≤ delivery_date

PostgreSQL



Quality Gate 2

Schema enforcement

- Column types
  Integer, Date, Text
- NOT NULL
  required columns
- NUMERIC(15,2)
  money columns
- PRIMARY KEY
  no duplicate IDs

dbt



Quality Gate 3

Model tests

- unique
  17 tests on key columns
- not_null
  26 tests
- Run by dbt build
  each model tested after build
- Visible in Dagster
  reported as asset checks

Dagster



Quality Gate 4

Asset checks

- reject_rate < 20%
  all 13 ingestion assets
- row_count > 0
  4 unpartitioned tables
- Run metadata
  rows read, valid, rejected
- Failure alert
  email on run failure

https://docs.dagster.io/integrations/libraries/dbt/reference

Deployment

❖ Retry policy



Retry Policy in Dagster: A declarative policy to attach, which will have retries requested on exception.

```
retry_policy=dg.RetryPolicy(max_retries=3, delay=30, backoff=dg.Backoff.EXPONENTIAL)
```



Where is this applied in the **project**?

Location	File path	Retry policy
9 asset ingestion partitioned	defs/assets_ingest_partitioned.py	max_retries=3, delay=30, EXPONENTIAL
4 asset ingestion unpartitioned	defs/assets_ingest_dimension.py	max_retries=3, delay=30, EXPONENTIAL
3 model dbt incremental	defs/dbt_incremental.py	max_retries=2, delay=30, EXPONENTIAL
2 model dbt (remaining)	defs/dbt/defs.yaml	None

https://docs.dagster.io/guides/build/ops/op-retries

Deployment

❖ Retry policy

Test: intentionally set an incorrect `connection_url` via `--config-json` (without modifying any files).

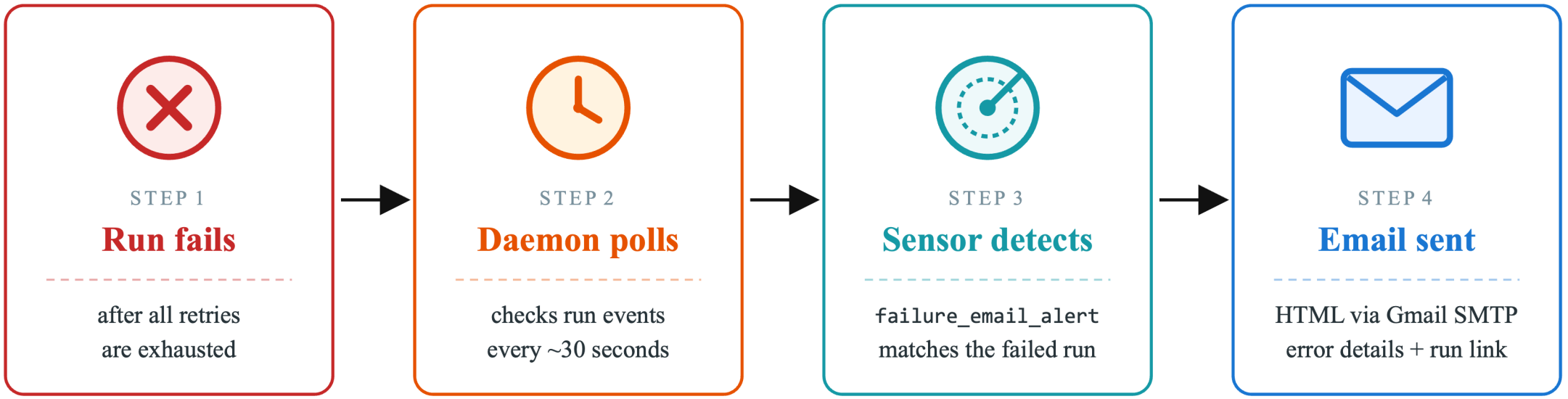


Deployment

❖ Integrate sensors for alerting

Sensors in Dagster: Sensors enable you to **take action in response to events** that occur either internally within Dagster or in external systems.

Current Project

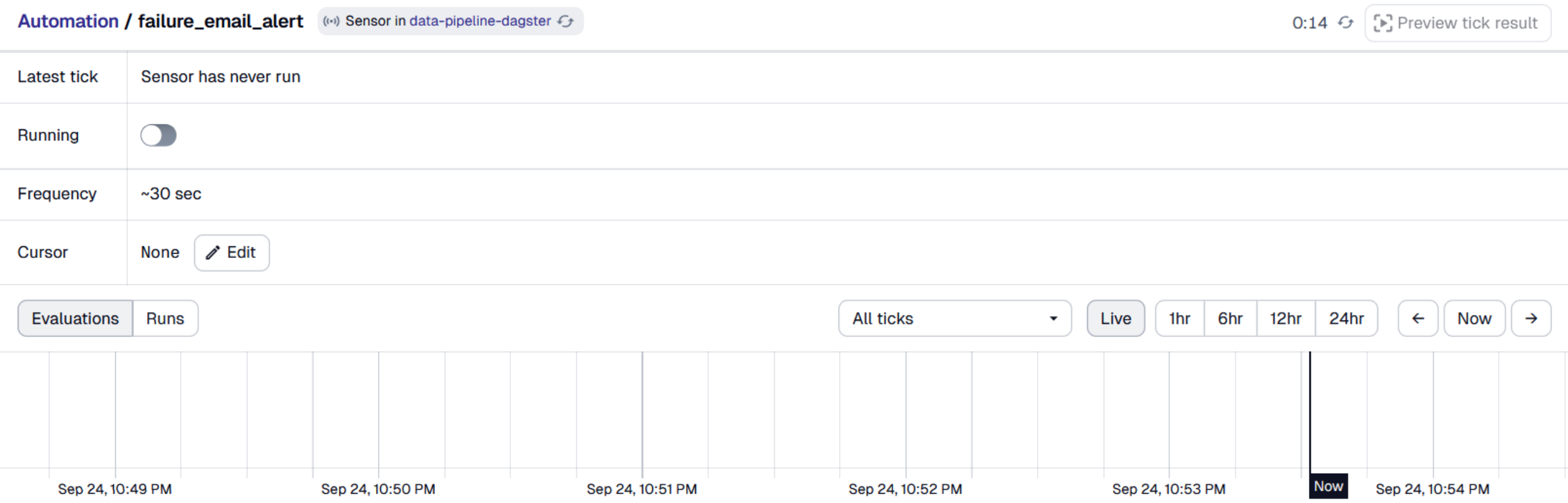


Requires: sensor started and dagster-daemon running.
A transient error that a retry fixes sends no email.

Deployment

❖ Integrate sensors for alerting

Current Project – Dagster WebUI



Deployment

❖ Integrate sensors for alerting

```
return dg.make_email_on_run_failure_sensor(
    email_from=GMAIL_USER,
email_password=GMAIL_APP_PASSWORD,
email_to=[ALERT_EMAIL_TO],
    email_body_fn=_failure_email_body,
email_subject_fn=_failure_email_subject,
    name="failure_email_alert",
monitor_all_code_locations=True,
)
```



[Dagster] __ASSET_JOB [2013-03-07] failed Hộp thư đến ×

✦ Tóm tắt email này

[redacted]

đến tôi

Dagster run failed

Job	__ASSET_JOB
Run ID	b8cdc2ab-5c57-415f-9f89-0b21142095b1
Partition	2013-03-07

Failed steps

Step	Error
raw__raw_orders	Exceeded max_retries of 3 Stack Trace: File "D:\dagster_data_pipeline\data-pipeline-dagster\.venv\Lib\site-packages\dagster_core\execution\plan\execute_plan.py", line 243, in dagster_event_sequence_for_step yield from check.generator(step_events) File "D:\dagster_data_pipeline\data-pipeline-dagster\.venv\Lib\site-packages\dagster_core\execution\plan\execute_step.py", line 505, in core_dagster_event_sequence_for_step for user_event in _step_output_error_checked_user_event_sequence(~~~~~~^ step_context, ^^^^^^^^^^^^^ _process_asset_results_to_events(step_context, user_event_sequence),

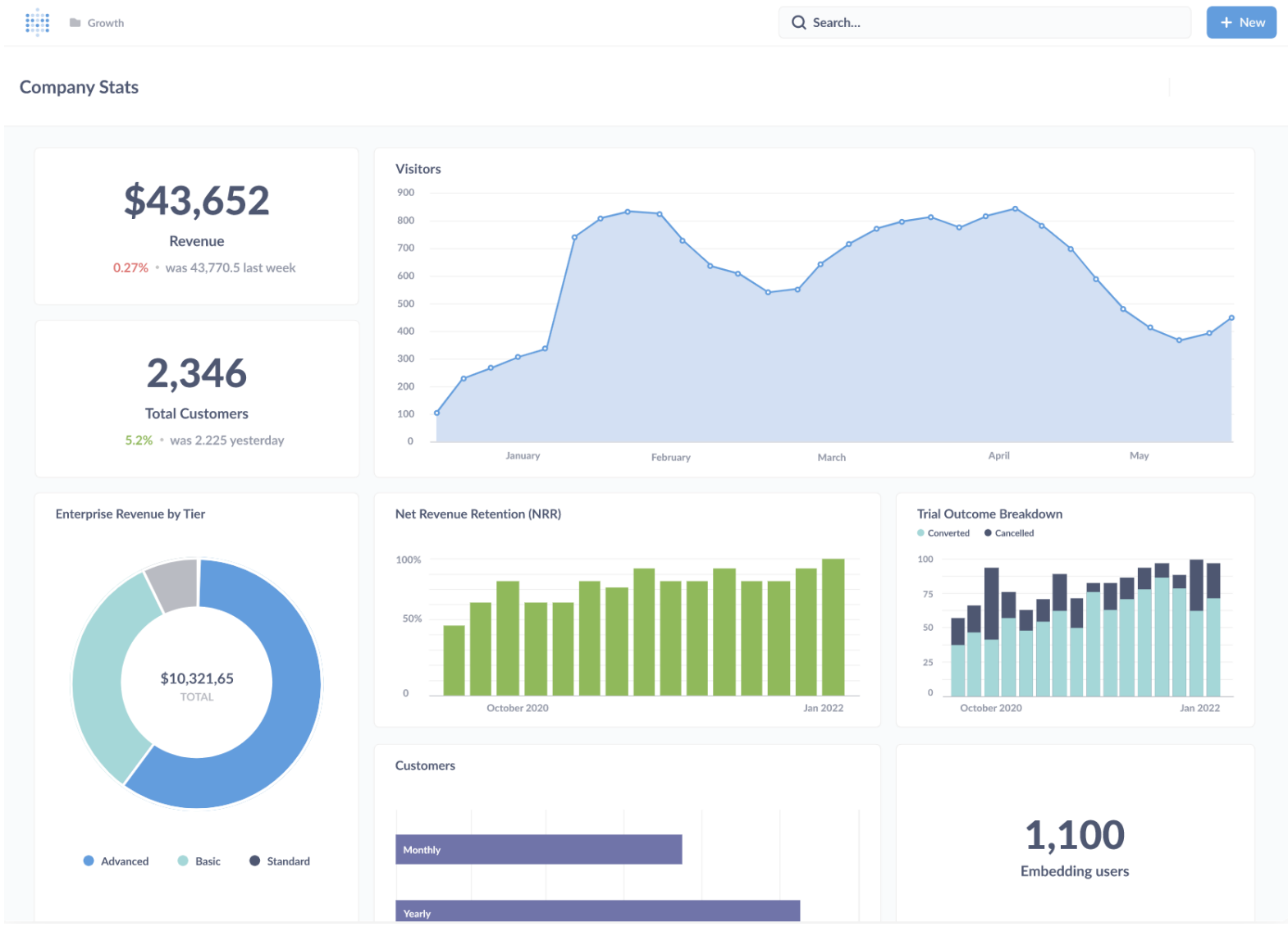
View run in Dagster UI

Deployment

❖ Visualizing the marts

Metabase

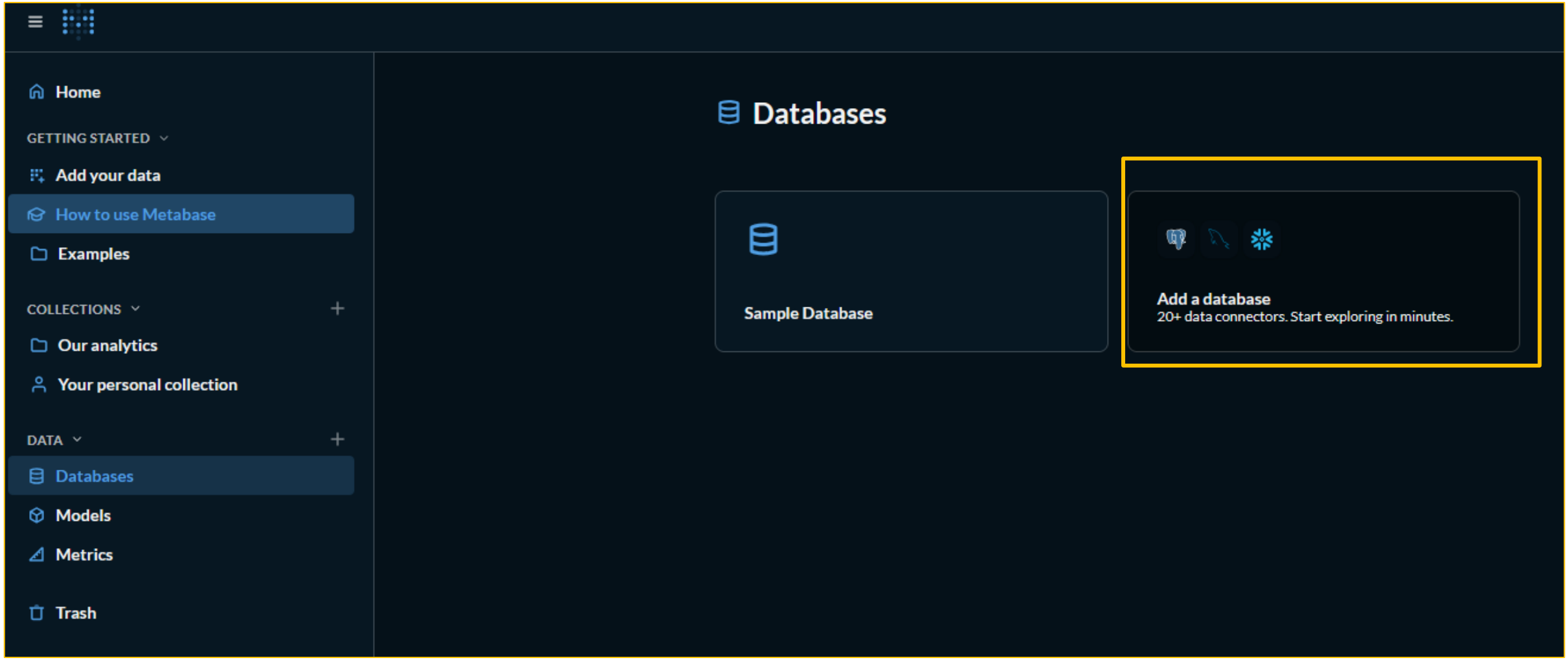
- Metabase is an open-source business intelligence platform.
- You can use Metabase to ask questions about your data, or embed Metabase in your app to let your customers explore their data on their own.



https://www.metabase.com/docs/latest/

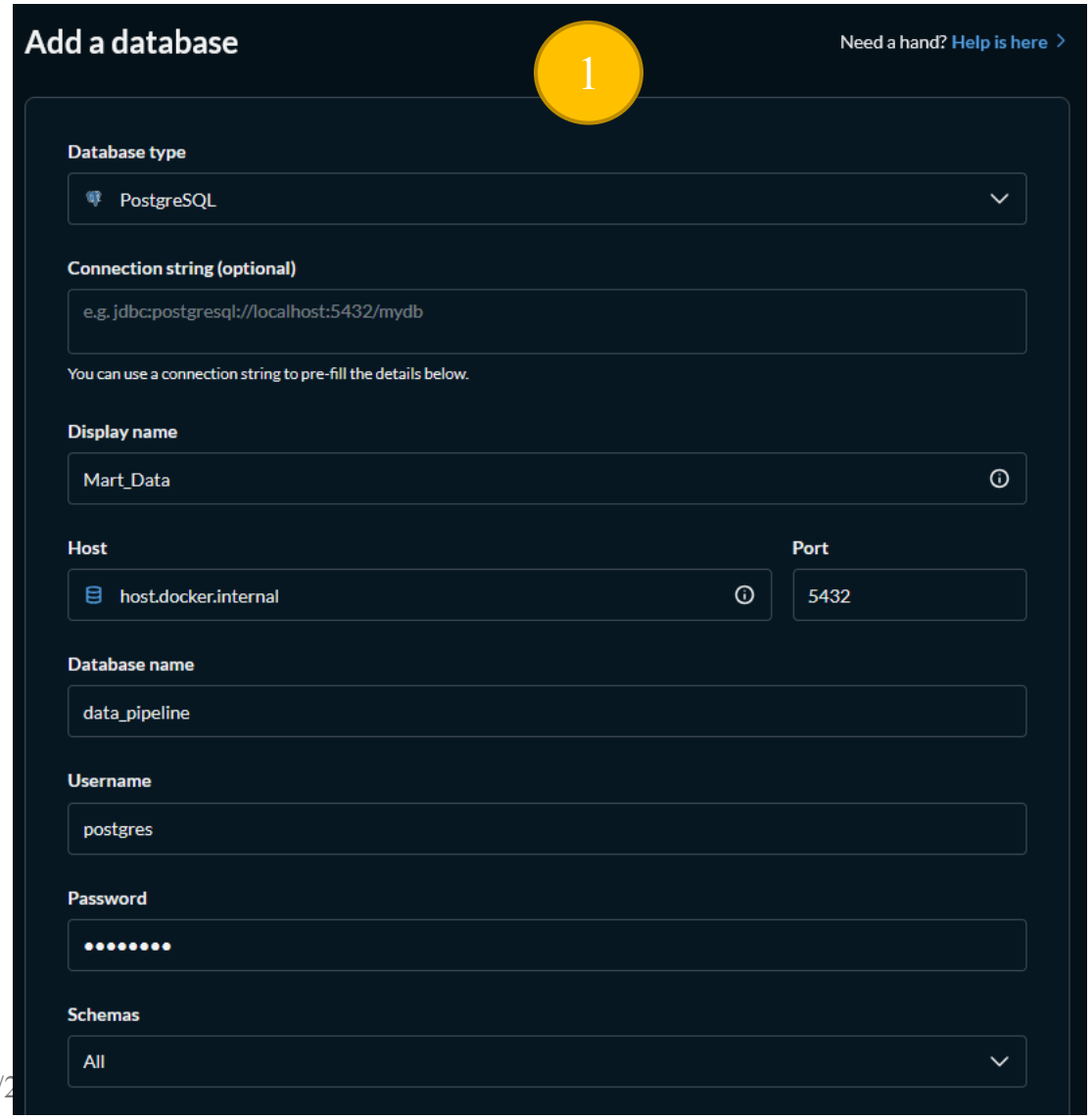
Deployment

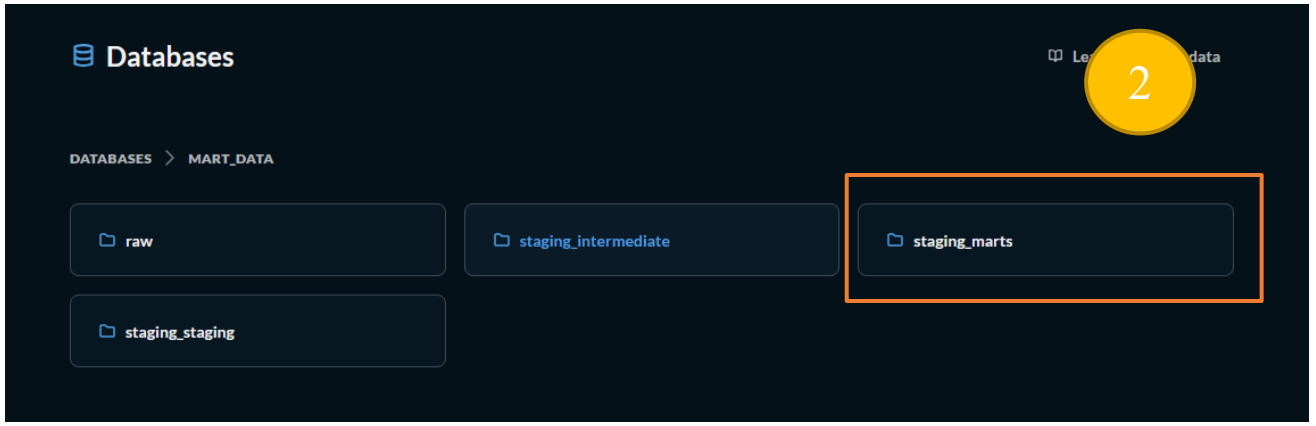
❖ Set up Metabase

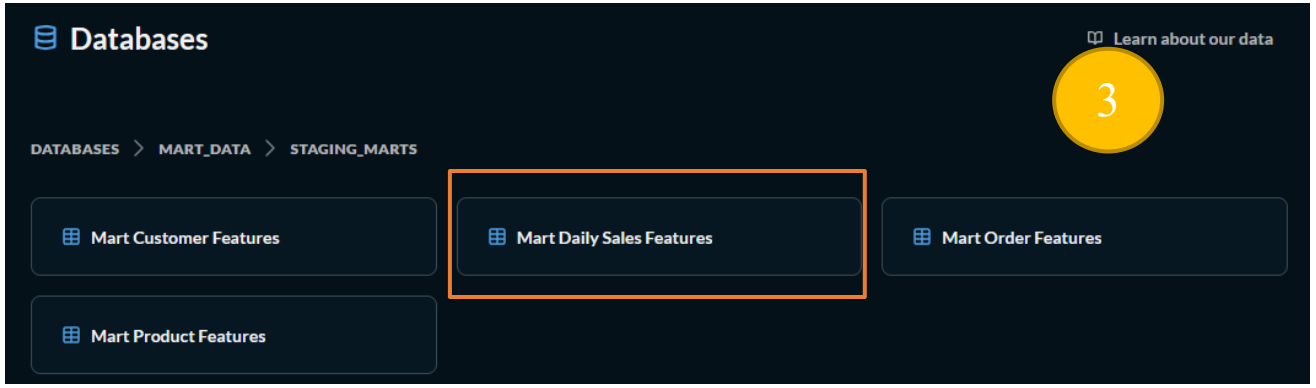


Deployment

❖ Visualizing the marts

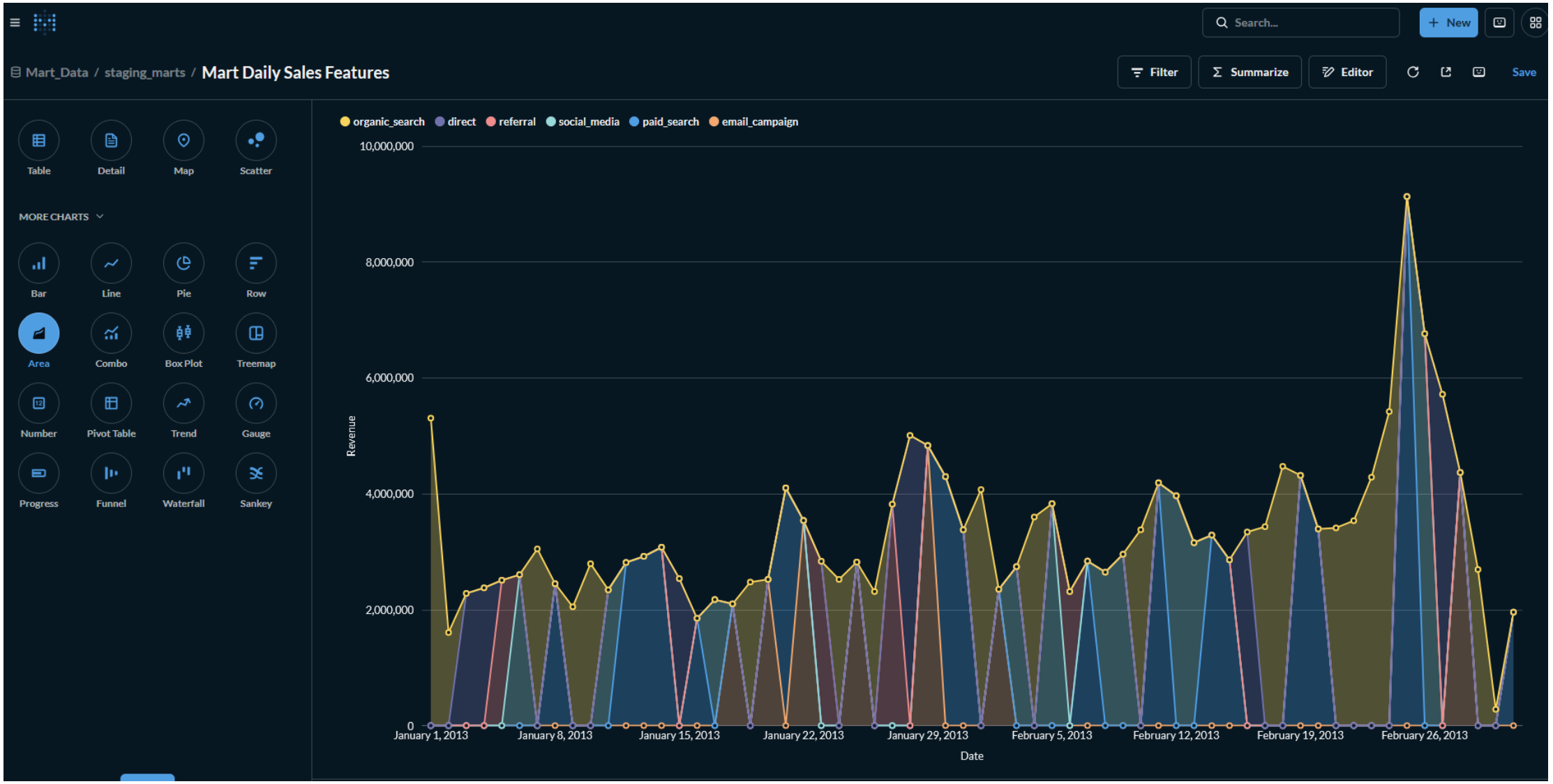




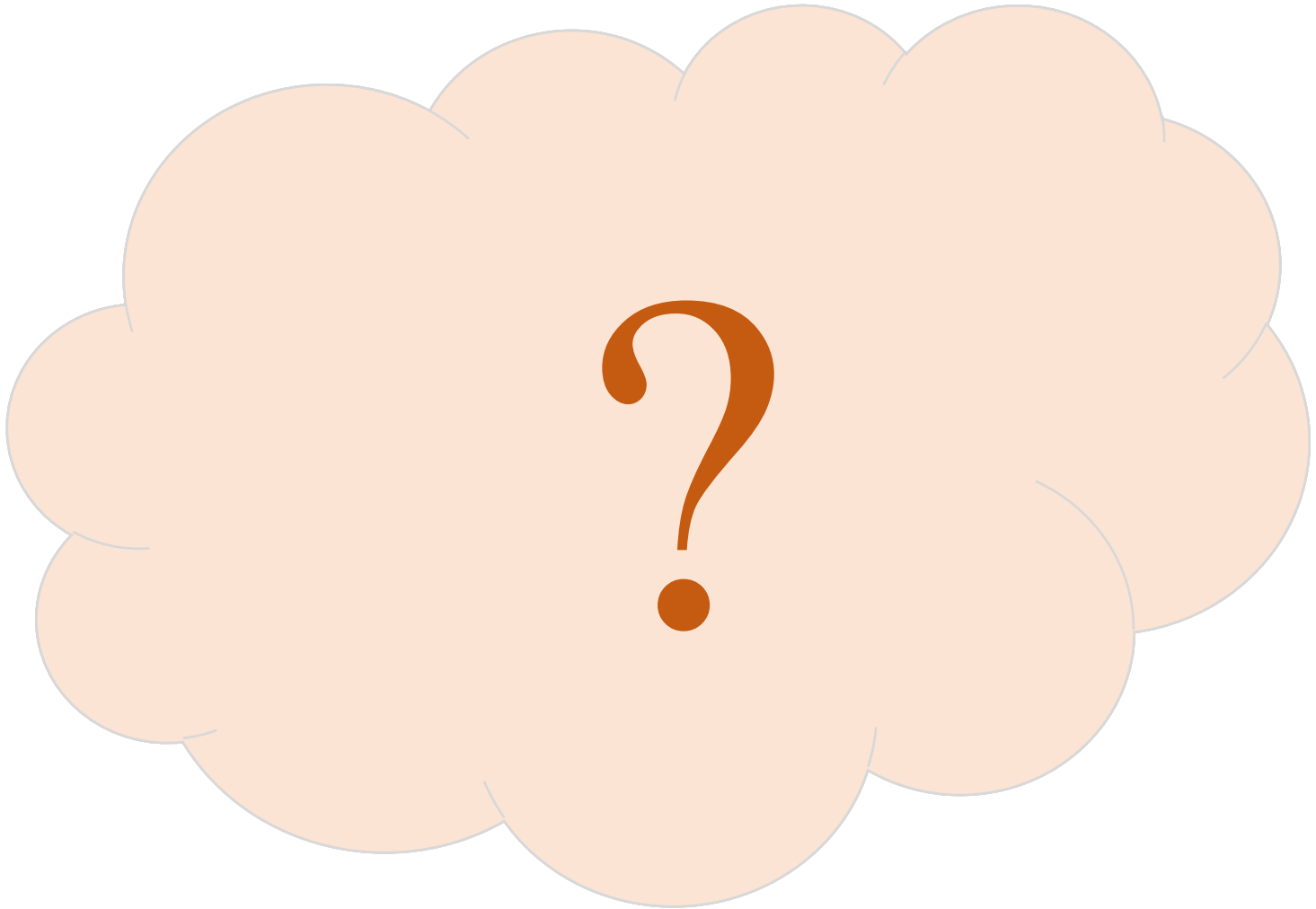


Deployment

❖ Visualizing the marts



Question



References

- https://www.fivetran.com/blog/10-data-pipeline-challenges-your-engineers-will-have-to-solve
- https://cribl.io/blog/the-observability-pipeline/
- https://www.6sigma.us/six-sigma-in-focus/data-quality-assurance/
- https://docs.dagster.io/


Thank you!